

CARD: Robustness-Preserving Transfer Learning for Network Intrusion Detection via Contrastive Adversarial Representation Distillation

Mengdie Huang¹, Student Member, IEEE, Yingjun Lin², Ninghui Li², Fellow, IEEE, Xiaofeng Chen¹, Fellow, IEEE, and Elisa Bertino³, Fellow, IEEE

Abstract—Robust neural networks are essential to build network intrusion detection systems resilient to evasion attacks. Learning such models via adversarial training demands extensive labeled data and high model capacity, making it impractical in evolving, resource-constrained threat environments. Transfer learning (TL) uses pre-trained models to enhance downstream tasks, offering a promising mitigation approach. However, most TL approaches prioritize performance on clean examples without addressing robustness against adversarial examples and random variations. Our empirical study reveals that standard fine-tuning and distillation often yield accurate but not robust models, while the few existing adversarial TL provide limited robustness. In this article, we propose a novel robustness-preserving TL framework, Contrastive Adversarial Representation Distillation (CARD), to generate a robust target model by transferring robustness and performance from a robust source model into the target task. CARD tackles three issues: (i) target domain data scarcity; (ii) differences in data domains and model architectures between target and source tasks; and (iii) target model robustness against static and adaptive evasion attacks, and natural corruptions. Experiments on binary and multiclass detection show that CARD outperforms state-of-the-art methods in various TL tasks across data domains and model architectures when only 5% training data is available, achieving 17.67% and 8.38% higher adversarial robust accuracy as well as 9.75% and 11.42% higher natural robust accuracy than adversarial fine-tuning and distillation.

Index Terms—Neural network, transfer learning, robustness, evasion attack, natural corruption, network intrusion detection.

I. INTRODUCTION

MACHINE learning, particularly deep learning (DL), has become increasingly attractive for developing network intrusion detection (NID) systems. This is largely due to the

exceptional capability of deep neural networks to uncover hidden representations in vast datasets and enhance classification accuracy, as evidenced in computer vision, speech recognition, and networking. In this work, we focus on NID models based on deep neural networks (DNNs) for their effectiveness in handling high-dimensional features [1], scalability with large datasets [2], and adaptability to changing network threats [3].

When large labeled datasets are unavailable or resources for model deployment are constrained, training high-performance classifiers from scratch becomes difficult [4]. In particular, achieving adversarial robustness through adversarial training from scratch requires significantly more training samples [5], [6] and higher model capacities [7], [8], [9], [10]. In NID, complex and evolving intrusion patterns, along with privacy concerns, often hinder the collection and labeling of sufficient traffic data. Meanwhile, real-world environments like IoT and mobile edge computing often require lightweight models, inevitably sacrificing the representation capabilities of DNNs.

Transfer learning (TL) is a technique for tackling data scarcity, in which a pre-trained model, called the *source model*, developed for a *source domain* task with abundant data, is used as the starting point for learning a model, called the *target model*, for a different but related *target domain* task with limited training data [4]. Domain refers to the entire data distribution, including its feature space and label space. In TL, we also consider a target task with a different model architecture, known as distillation [11]. By leveraging knowledge gained from a *source task* to improve performance on a *target task*, TL has been successfully applied in image [12], natural language [13], and network traffic [14], [15]. TL is also pivotal for foundation models (FMs) [16], which are large models trained on massive datasets. Previous research in NID shows that TL based on fine-tuning greatly improves and accelerates the training when target domain labeled data are scarce [17] while distillation-based TL makes small target models suitable for devices with limited storage resources [18], [19]. However, existing TL approaches for NID do not address robustness against model evasion attacks, where attackers manipulate inputs as adversarial examples to induce misclassification.

Motivation: Recently, in the image field, a few studies on TL, including adversarial finetuning [6], [7] and distillation [9], [20], of robust classifiers pretrained adversarially on larger datasets have demonstrated the feasibility of transferring robustness across tasks. However, (i) these approaches rely on a suitably large target domain dataset for training, neglecting the challenge of limited data availability. Also, (ii) they focus only on TL

Received 8 June 2024; revised 9 March 2025; accepted 9 April 2025. Date of publication 18 April 2025; date of current version 4 September 2025. The work of Xiaofeng Chen was supported in part by the National Natural Science Foundation of China under Grant 62432012 and Grant 62121001 and in part by the 111 Center under Grant B16037. (Corresponding author: Xiaofeng Chen.)

Mengdie Huang is with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710126, China, and also with the Department of Computer Science, Purdue University, West Lafayette, IN 47906 USA (e-mail: mdhuang1@stu.xidian.edu.cn).

Yingjun Lin, Ninghui Li, and Elisa Bertino are with the Department of Computer Science, Purdue University, West Lafayette, IN 47906 USA (e-mail: lin1368@purdue.edu; ninghui@purdue.edu; bertino@purdue.edu).

Xiaofeng Chen is with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710126, China (e-mail: xfchen@xidian.edu.cn). Digital Object Identifier 10.1109/TDSC.2025.3562600

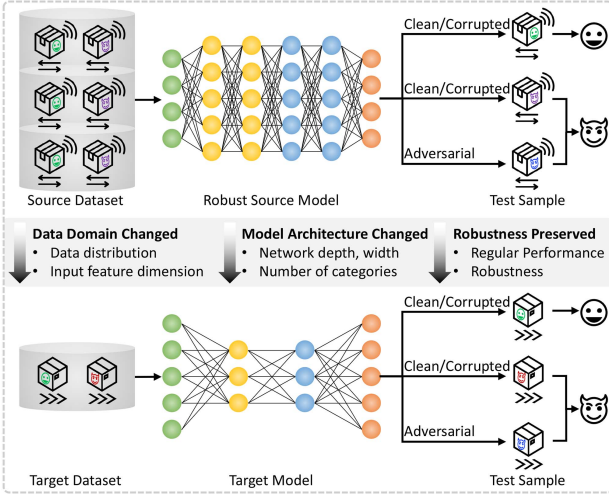


Fig. 1. Transfer learning scenarios where CARD can be applied to. The target task and the source task differ in data domains, model architectures, or both.

scenarios with a single difference between source and target tasks, such as fine-tuning for cross-domain TL or distillation for cross-model TL, making them unsuitable when both data domains and model architectures differ. Research addressing the following crucial problems in the network traffic field remains lacking. *Given* a robust source NID model f^S trained to be robust against source domain evasion attacks, let f^T be a target NID model derived from f^S using TL, **i** does standard TL make f^T automatically robust against target domain evasion attacks? **ii** if not, how can TL be designed to transfer robustness from f^S to f^T in data scarcity scenarios? **iii** will the expansion of available data lead to significant fluctuations in the performance of robustness-preserving TL?

Overview: In this paper, we address these questions. First, our empirical study shows that robustness cannot be effectively transferred to the target NID model from the *robust source model* through standard fine-tuning or distillation. Then, we present the first robustness-preserving TL method designed for NID that ensures the robustness and clean performance of the target model. Specifically, we propose Contrastive Adversarial Representation Distillation (CARD), a novel TL framework to generate robust target models from robust source models with limited target domain data. CARD supports three TL scenarios (see Fig. 1): source and target tasks **①** differ only in data domains (e.g., changes in input feature dimensions, marginal data distribution, label spaces); **②** differ only in model architectures (e.g., changes in depth, width, or building blocks); **③** differ in both. CARD simultaneously addresses three problems: (i) target domain data scarcity; (ii) differences in both data domains and model architectures between target and source tasks; and (iii) target model robustness against both static and adaptive evasion attacks, and natural corruptions.

Our Method: The design of CARD is based on three key elements: **①** To capture and transfer robustness knowledge from the source model, we propose a Dual Robustness-aware Contrastive View Construction strategy based on the adversarial manipulation view and the natural corruption view. Such a strategy optimizes the target model towards the goal of making these two types of input variants closer to the clean input (anchor) in the hidden representation space, while distancing them from samples of other classes, *thereby generating a target*

model robust to target domain adversarial examples and natural corruptions. **②** To support TL across data domains and model architectures, we develop a Dual Adaptive Dimension Alignment mechanism. This mechanism uses three learnable embedding networks to align input feature dimensions of the target domain with the source domain, and align hidden representation dimensions of the target model with the source model, *thereby adaptively generating embedded target domain features that can be fed to the input layer of the source model and embedded hidden representations that can be used for similarity measurement between different models.* **③** To ensure clean and robust performance of the target model with limited target domain training samples, we design a comprehensive loss including representation contrastive learning, output distribution distillation, and adversarial classification terms for contrastive adversarial representation distillation. This loss encourages the target model to mimic universal robust representations of the source model trained on sufficient source domain data and emphasize locally clean and adversarial patterns from the target domain, *thereby guiding the target model to converge to the underlying data distribution with limited target domain labeled training data.*

Experiments: We thoroughly evaluated CARD and TL baselines on NID tasks in three TL scenarios—TL across data domains, across model architectures, and across both—under (i) data scarcity and (ii) data expanding settings, respectively. Each task covers binary (benign versus malicious) and multiclass detection (fine-grained intrusion types) versions. Specifically, comparative baselines of CARD, the first robustness-preserving TL work targeting NID models, include not only standard TL methods that do not address robustness, such as NID-specific fine-tuning [17] and distillation [18], but also state-of-the-art (SOTA) image-specific robustness-preserving TL methods, such as adversarial fine-tuning (e.g., FRFE [6], TWINS [21]) and adversarial distillation (e.g., VAD [20], AAD [22]).

Contributions: In summary, we make three contributions:

- We propose CARD, the first robustness-preserving TL framework for NID, designed to generate a robust target model with limited training data by contrastively distilling a robust source model. It is the first solution that supports transfer robustness across both data domains and model architectures.
- We first introduce the novel notion of dual robustness-aware views in contrastive distillation, including the adversarial manipulation view and natural corruption view, which facilitates capturing and transferring underlying robust knowledge from the hidden representation space of the source model.
- We extensively evaluate the robustness and clean performance of the target model. (i) In data-scarce settings, averaged results show that CARD improves robust accuracy by 13% on adversarial examples, 11% on corrupted examples, and clean accuracy by 6% over SOTA adversarial fine-tuning and distillation baselines. (ii) In data-expanding settings, CARD demonstrates stable, monotonically increasing performance and sustained robustness advantage, further confirming its effective generalization with limited data.

II. RELATED WORK

A gap in transfer learning (TL) and robust DL is understanding whether robustness, if present in the source model, is automatically inherited by target models derived from it. If so,

one could train a robust foundation model (e.g., via adversarial training on a large source domain dataset [5], [23]), and the cost of this robust training would be amortized by target domains, as target models would inherit robustness through robustness-preserving TL without extra expensive robust training. Recently, robustness transfer from an adversarially trained model has been investigated for image data. However, it remains unexplored in network traffic classification.

A. Standard Transfer Learning

Conventional TL, known as standard TL, aims to transfer knowledge from a **source model** f^S to a **target model** f^T to improve the clean performance and learning efficiency of the target task, especially when the **target domain** $\mathcal{D}^T$ has limited data [4]. In this work, we focus on both TL across data domains (e.g., fine-tuning) and TL across model architectures (e.g., distillation), which have also been explored in NID.

1) *Standard Fine-Tuning*: Fine-tuning (FT) adapts a model f^S pretrained in the **source domain** $\mathcal{D}^S$ to $\mathcal{D}^T$ by updating parameters in a layer-wise manner, such as retraining the early representation learning layers [24], later classification layers [17] or the entire model. In the area of NID, Singla et al. [17] demonstrated the viability of TL for NID by fine-tuning the final softmax layer of a DNN with minimal new labeled data. Xu et al. [25] analyzed the impact of fine-tuning different layers to adapt NID models to rapid changes. Andresini et al. [24] fine-tuned two autoencoders and a Conv1D layer of a NID model on refined input-output pairs for drift detection. FT assumes that limited labeled data in $\mathcal{D}^T$ is available and that $\mathcal{D}^S$ and $\mathcal{D}^T$, while distinct, share some correlations in the data features. Thus, FT is suitable for cross-domain TL while maintaining the architecture of f^S .

2) *Standard Distillation*: Knowledge distillation (KD) [11] aims to compress f^S to generate f^T by transferring knowledge from a large, complex model (teacher) to a smaller, simpler model (student), usually within the same domain and task. f^T can learn from f^S by mimicking output logits [26], soft labels [11], hard labels [19], or intermediate representations [27] of f^S . In the area of NID, Zhao et al. [18] used soft label-guided KD for producing a lightweight separable convolutions-based NID model. Gao et al. [28] improved federated NID accuracy by selecting the top two nodes as teachers and distilling their performance to other nodes using soft labels. Others [19] compressed the NID teacher into a lightweight model for deployment using both soft and hard labels. Thus, KD is ideal for lightweight-oriented cross-model TL, with unchanged feature space and label space.

B. Robustness-Preserving Transfer Learning

Robustness-preserving TL seeks to maintain or enhance the robustness of a robust source model f^S in the target model f^T during knowledge transfer between tasks. Existing techniques include adversarial fine-tuning and adversarial distillation.

1) *Adversarial Fine-Tuning*: Adversarial fine-tuning (AFT) adapts a robust f^S to a target domain $\mathcal{D}^T$ while preserving robustness against target domain evasion attacks, typically by incorporating adversarial examples in standard FT. Existing work has primarily been tested in TL tasks from ImageNet to CIFAR. Hendricks et al. [7] demonstrated that *adversarially fine-tuning* the last classification layer of an adversarially pre-trained f^S using adversarial examples achieves better robustness

and similar clean performance on $\mathcal{D}^T$ than adversarial training f^T from scratch. Shafahi et al. [6] proposed a technique to train a robust f^T by fine-tuning the classification layers on top of the Frozen Robust Feature Extractor (FRFE) of a robust f^S . They found that *AFT using only target domain clean data results in better clean performance but lower robustness than AFT using only adversarial examples*. Fan et al. [29] introduced adversarial contrastive learning to explore AFT within self-supervision using unlabeled data. Recently, Liu et al. [21] developed TwoWing Normalization (TWINS) consisting of a frozen net and an adaptive net to preserve the robustness of f^S in $\mathcal{D}^T$, where the means and standard deviations from the frozen normalization layer of f^S and updated normalization layer of f^T are jointly used for AFT.

2) *Adversarial Distillation*: Adversarial distillation (AD) distills a large, robust f^S into a smaller, robust f^T for the same task by incorporating adversarial examples in standard KD. Goldblum et al. [20] found that *robustness in f^S can be inherited by f^T when distilled using clean examples and more when using adversarial examples*. They then introduced adversarially robust distillation, referred to as Vanilla Adversarial Distillation (VAD), to minimize KL divergence between the output distribution of f^S over adversarial examples and that of f^T over clean examples. Muhammad et al. [30] proposed to distill hidden activation maps of f^S using mixed examples. Zhu et al. [31] designed introspective adversarial distillation (IAD) to encourage f^T to selectively mimic the behavior of f^S . Zi et al. [9] proposed Robust Soft Label Adversarial Distillation (RSLAD), guiding predictions of f^T on both clean and adversarial examples to mimic soft labels of f^S on clean examples. Maroto et al. [32] used a weighted combination of truth label and output of f^S as the supervision label for f^T on adversarial examples. Bai et al. [33] introduced contrastive learning to AD to generate image classifiers. Recently, Huang et al. [22] developed Adaptive Adversarial Distillation (AAD), using dynamically updated adversarial examples to boost AD.

C. Novelty of Our Method

Our work differs from existing approaches in these aspects:

- *Goal*: They focus on TL scenarios across only data domains (AFT) or only model architectures (AD), while ours extends applicability to the more challenging TL across both.
- *Threat Model*: They emphasize adversarial robustness to evasion attacks while we incorporate natural robustness to natural corruptions in addition to adversarial robustness.
- *Technique and Evaluation*: They are either standard TL for NID or single adversarial robustness transfer for images, evaluated on a suitable large training dataset, while we introduce first robustness-preserving TL for NID, evaluated on limited and expanding training datasets, respectively.

III. PRELIMINARIES

This section covers robustness-preserving TL preliminaries, focusing on adversarial fine-tuning and distillation. In what follows, $\mathcal{D}^S$ and $\mathcal{D}^T$ denote training datasets from source domain $\mathcal{D}^S$ and target domain $\mathcal{D}^T$, respectively.

A. Transfer Learning Based on Fine-Tuning

1) *Standard Fine-Tuning*: Fine-tuning (FT) adapts a pre-trained **source model** f^S to $\mathcal{D}^T$ by optimizing all or only

some layers, yielding a **target model** $f_{\theta}^T = f^S + \Delta\theta$ with the same architecture but updated parameters. The target model $f_{\theta}^T = f_{\theta_c}^T(f_{\theta_f}^T)$ can be split into a feature extractor $f_{\theta_f}^T$, typically using convolutional layers for representation learning, and a classifier $f_{\theta_c}^T$, employing fully connected layers for prediction.

a) *Linear FT*: $f_{\theta_c}^T$ is fine-tuned while $f_{\theta_f}^T$ is frozen [17], a.k.a. linear probing or linear evaluation, as defined in (1). y is the ground truth label of the **clean training sample** $x \in D^T$.

$$\min_{\theta_c} \mathbb{E}_{(x,y) \sim D^T} [\mathcal{L}(f_{\theta_c}^T(f_{\theta_f}^T(x)), y)] \quad (1)$$

b) *Full FT*: All model parameters θ are optimized to minimize the loss function $\mathcal{L}$, as defined in (2).

$$\min_{\theta} \mathbb{E}_{(x,y) \sim D^T} [\mathcal{L}(f_{\theta}^T(x), y)] \quad (2)$$

Different FT methods balance computational cost and adaptation differently, making them suitable for various scenarios.

2) *Adversarial Fine-Tuning*: Adversarial fine-tuning (**AFT**) aims to adapt a *robust* f^S to D^T , typically by incorporating **adversarial examples** $x^* = x + \delta^*$, which are obtained by manipulating x with adversarial perturbations $\delta^* = \arg \max_{\|\delta\|_p < \epsilon} \mathcal{L}(f_{\theta}^T(x + \delta), y)$, in standard FT to transfer adversarial robustness against evasion attacks from f^S to f_{θ}^T .

a) *Linear AFT*: Vanilla AFT methods [7], [34], [35] fine-tune the last classification layer using adversarial examples while freezing the robust feature extractor, as defined in (3).

$$\min_{\theta_c} \mathbb{E}_{(x,y) \sim D^T} [\mathcal{L}(f_{\theta_c}^T(f_{\theta_f}^T(x^*)), y)] \quad (3)$$

b) *Full AFT*: The entire pre-trained robust model is fine-tuned using adversarial examples from D^T [29], [36] as in (4).

$$\min_{\theta} \mathbb{E}_{(x,y) \sim D^T} [\mathcal{L}(f_{\theta}^T(x^*), y)] \quad (4)$$

FRFE [6] further explored full AFT using only clean samples from D^T with a learning without forgetting mechanism as in (5).

$$\min_{\theta = \theta_f \cup \theta_c} \mathbb{E}_{(x,y) \sim D^T} \mathcal{L}(f_{\theta}^T(x), y) + \|f_{\theta_f}^T(x) - f_{\theta_f}^S(x)\|_2, \quad (5)$$

Afterwards, a variant of FRFE with adversarial examples, as defined in (6), is used as an AFT baseline in TWINS [21].

$$\min_{\theta = \theta_f \cup \theta_c} \mathbb{E}_{(x,y) \sim D^T} \mathcal{L}(f_{\theta}^T(x^*), y) + \|f_{\theta_f}^T(x^*) - f_{\theta_f}^S(x^*)\|_2, \quad (6)$$

TWINS [21] splits D^T into two subsets D_l^T and D_u^T of equal size, and freezes parameters θ_b of the batch normalization layer in $f_{\theta_f}^S$ when performing AFT on half of D^T , as defined in (7).

$$\min_{\theta_f \cup \theta_c} \mathbb{E}_{(x_l, y) \sim D_l^T, (x_u, y) \sim D_u^T} [\mathcal{L}(f_{\theta}^T(x_l^*), y) + \mathcal{L}(f_{\theta_f/\theta_b}^T(x_u^*), y)] \quad (7)$$

B. Transfer Learning Based on Distillation

1) *Standard Distillation*: Knowledge distillation (KD) transfers knowledge from a **source model** f^S (teacher) to a smaller **target model** f_{θ}^T (student) using clean training samples x within the same dataset $D^S = D^T$ and label space.

a) *Logit-Based KD*: f_{θ}^T learns the output probability distribution o^T from the output distribution o^S of f^S using softened logits and hard labels [18], as defined in (8).

$$\min_{\theta} \mathbb{E}_{(x,y) \sim D^T} [\tau^2 \text{KL}(o^{S^T}(x) | o^{T^T}(x)) + \mathcal{L}(f_{\theta}^T(x), y)] \quad (8)$$

where KL denotes KL divergence, $o^S(x)$ and $o^T(x)$ are output distributions of f^S and f_{θ}^T on x , and τ is the temperature to

scale the distribution via softmax S , as defined in (9).

$$o^{T^T}(x) = S(f_{\theta}^T(x)/\tau), o^{S^T}(x) = S(f^S(x)/\tau), \quad (9)$$

b) *Feature-Based KD*: f_{θ}^T mimics the intermediate features of f_{θ}^S obtained by the representation learner $f_{\theta_r}^S$ to learn hidden representations $f_{\theta_r}^T(x)$ of $f_{\theta_r}^T$ on D^T , as defined in (10).

$$\min_{\theta} \mathbb{E}_{(x,y) \sim D^T} \|\rho(f_{\theta_r}^T(x)) - \rho(f_{\theta_r}^S(x))\|_p, \quad (10)$$

where ρ denotes the transformation of neuron activation [27].

2) *Adversarial Distillation*: Adversarial Distillation (**AD**) aims to compress a *robust* f^S into a smaller f_{θ}^T , usually by involving adversarial examples $x^* = x + \delta^*$ in standard KD, to preserve robustness against evasion attacks from f^S in f_{θ}^T .

a) *Singular Sample-Guided AD*: Vanilla adversarial distillation (VAD) [20] minimizes the difference between the probability distribution $o^{T^T}(x^*)$ output by f_{θ}^T on x^* and $o^{S^T}(x)$ output by f^S on x , as defined in (11), where $\delta^* = \arg \max_{\|\delta\|_p < \epsilon} \mathcal{L}(f_{\theta}^T(x + \delta), y)$. In this case, soft labels $o^{S^T}(x) = S(f^S(x)/\tau)$ of f^S on clean samples x are used as the sole supervision for true distribution in KL loss.

$$\min_{\theta} \mathbb{E}_{(x,y) \sim D^T} [\tau^2 \text{KL}(o^{S^T}(x) | o^{T^T}(x^*)) + \mathcal{L}(f_{\theta}^T(x), y)], \quad (11)$$

RSLAD [9] replaces the hard label y in the loss of VAD with logits $f^S(x)$ of f^S on x and fixed $\tau = 1$, as defined in (12).

$$\min_{\theta} \mathbb{E}_{(x,y) \sim D^T} [\text{KL}(o^S(x) | o^T(x^*)) + \text{KL}(o^S(x) | o^T(x))], \quad (12)$$

b) *Dual Samples-Guided AD*: In improved AD, soft labels output by f^S on both x and x^* are used for supervision in the KL loss. Among them, AAD [22] defined in (13), where $\delta^* = \arg \max_{\|\delta\|_p < \epsilon} \text{KL}(o^S(x^*) | o^T(x^*))$, was claimed to outperform VAD [20], RSLAD [9], and IAD [31] due to its dynamical optimization of adversarial examples.

$$\min_{\theta} \mathbb{E}_{(x,y) \sim D^T} [\text{KL}(o^{S^T}(x^*) | o^{T^T}(x^*)) + \text{KL}(f^S(x) | f_{\theta}^T(x))], \quad (13)$$

C. Adversarial Example and Adversarial Training

1) *Adversarial Example*: The attacker manipulates clean sample x as adversarial examples $x^* = x + \delta^*$ to cause misclassification of DNNs, where $\delta^* = \arg \max_{\|\delta\|_p < \epsilon} \mathcal{L}(f_{\theta}^T(x^*), y)$ is obtained via single-step [37] or iterative optimization [23].

2) *Adversarial Training*: The defender trains a model f_{θ} on both adversarial and clean examples from scratch to enhance robustness, minimizing the worst-case empirical loss $\min_{\theta} \mathbb{E}_{(x,y) \sim D} [\max_{\|\delta\|_p < \epsilon} \mathcal{L}(f_{\theta}(x^*), y) + (f_{\theta}(x), y)]$ [23].

IV. PROBLEM STATEMENT

A. Threat Model

We mainly consider robustness threats posed by evasion attacks and natural corruptions.

1) *Evasion Attack*: An attacker leverages adversarial examples to induce misclassification in DNN models [23]. For instance, **adversarial malicious traffic** generated from **clean malicious traffic** can significantly enhance the evasion success rate against DNN-based NID models [38], [39], [40]. We assume an evasion attacker who crafts adversarial examples from clean examples in the target domain to deceive the victim target model. In addition, we assume that the attacker has access to the architecture and parameters of the target model, creating

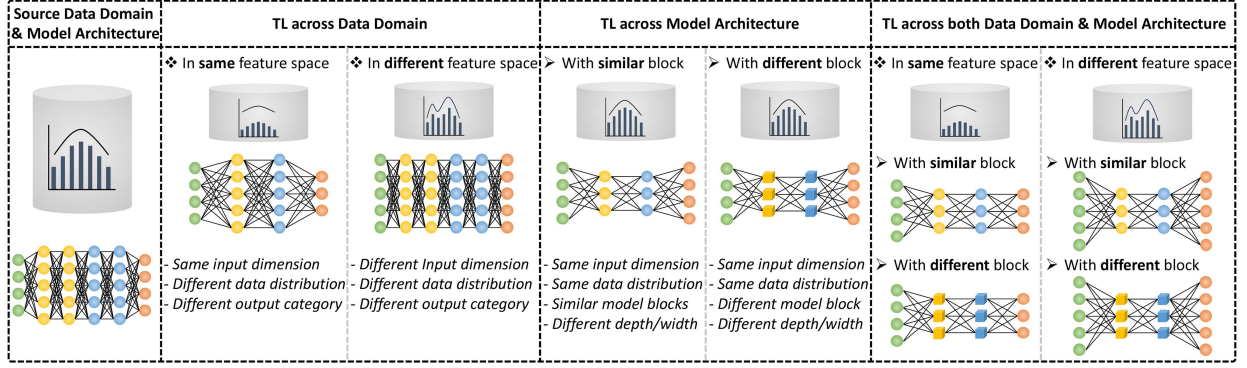


Fig. 2. Differences in data domains and model architectures between the target and source tasks in the transfer learning (TL) scenarios considered by CARD.

a high-threat scenario that enables the generation of effective adversarial examples and ensures a rigorous assessment of the target model's robustness in the context of TL.

It is important to note that in robustness-preserving TL, an attacker cannot easily infer the defense strategy of the target model, even with white-box access, as its robustness is derived partly from the source model and partly from adaptation to the target domain, rather than from a single explicit defense like adversarial training. In particular, attackers typically lack access to the robust source model in real-world settings, which is usually trained by parties with extensive resources and time. Furthermore, even with internal access to the source model, the adaptivity and complexity of a robustness-preserving TL strategy also make it difficult for the attacker to reconstruct the TL process exactly. Thus, in robustness-preserving TL, the hybrid and indirect robustness origins make it not easy for an attacker, even with access to the target model's parameters, to infer the complete defense algorithm used by the target model, such as the robust source model parameters and the TL strategy. So far, no adaptive evasion attack has addressed this, except for our exploration in the experimental evaluation.

2) *Natural Corruption*: It refers to a shift in the input feature distribution arising from natural factors, like environmental variations or noise [41], in data transmission or operating conditions, rather than intentional manipulation. For example, natural corruptions in images may arise from lighting and camera quality changes [41], while in speech data they could result from background noise and differences in recording devices [42]. In NID, natural corruption refers to changes in the statistical properties of network traffic features caused by variations in the network environment [40]. Given a clean input x , the corrupted sample $\tilde{x}$ is generated as in (14).

$$\tilde{x} = [x_t + \alpha_t \cdot \zeta_t, x_q + \alpha_q \cdot \zeta_q, x_l + \alpha_l \cdot \zeta_l], \quad (14)$$

where x_t , x_q , and x_l denote *time*-related, *quantity*-related, and *length*-related feature, respectively; ζ_t , ζ_q , and ζ_l are random noise vectors (e.g., Gaussian noise) for each feature group; and α_t , α_q , and α_l are scalar factors. Noise is added only to relevant features, leaving the rest unchanged. By introducing random noise into temporal and spatial characteristics, we aim to simulate natural corruptions from latency-induced changes in inter-arrival times and flow duration, fluctuations in packet number from network congestion, and variations in packet size due to hardware inconsistencies or protocol differences.

B. Research Goal

We aim to adapt the robustness and performance of a robust source model f^S to a target model f_θ^T that may differ in data domains or model architectures from the source task, using a small number of labeled samples. We focus on robustness-preserving TL in three scenarios for NID tasks (see Fig. 2)

1) *Robustness-Preserving TL Across Data Domains*: Ensure robustness transfer when using TL to generate a f_θ^T that differs from the robust f^S in data domains w.r.t input feature dimensions, marginal data distribution, or label space: For TL across only data domains (e.g., fine-tuning), the target and source tasks share the same model architecture but differ in their data domains. (i) In the context of NID, a common phenomenon is that the data distribution changes due to the emergence of new intrusion types, while the **input feature space** remains unchanged; that is, the number and meaning of the network traffic feature dimensions used by the detector do not change. (ii) Furthermore, when network protocols or service types change, the tools for processing network traffic features in the target task may differ from those in the source task. These differences in the input feature space between the target and source domains lead to a larger number of changes in data distribution. Thus, addressing robustness and performance transfer across data domains for NID tasks is critical, especially when there are insufficient labeled intrusion instances in the new task.

2) *Robustness-Preserving TL Across Model Architectures*: Ensure robustness transfer when using TL to yield a f_θ^T that differs from the robust f^S in model architectures w.r.t depth, width, or building blocks: For TL across only model architectures (e.g., distilling a large source model into a small model), the target and source tasks share the same data domain but differ in their model architectures. (i) In the context of NID, models must often be deployed in resource-constrained environments (e.g., IoT, edge computing, or mobile devices), so the high-performance source model needs to be compressed into a smaller target model with reduced capacity and storage overhead. (ii) In particular, training a small model from scratch is not ideal, as such a model typically cannot capture complex intrusion patterns. (iii) Furthermore, prior works have shown that adversarial robustness with good generalization is more easily obtained by adversarial training on high-capacity DNNs than on small models [7], [8], [9], [10], and requires large amounts of labeled samples [5], [6]. Thus, achieving robustness and performance transfer across model architectures is essential for NID with scarce labeled data.

3) *Robustness-Preserving TL Across Both Data Domains and Model Architectures*: Ensure robustness transfer when using

TL to produce a f_θ^T that differs from the robust f^S in both data domains and model architectures: This is a more complex situation where the target task not only undergoes a data distribution change but also requires a transition to a more lightweight model architecture. (i) In network security, adapting a large-scale robust source model trained on abundant source domain data to downstream tasks with limited target domain data and resources is crucial for the efficient production and practical deployment of NID systems. (ii) However, adversarial fine-tuning (AFT) cannot transfer knowledge across model architectures, while adversarial distillation (AD) fails across data domains. (iii) Furthermore, existing approaches are unable to balance robustness and clean performance, e.g., AFT and AD often degrades clean performance compared to standard methods [7], [20]. Thus, we need an TL approach that can preserve robustness across both data domains and model architectures while balancing clean performance.

C. Solution Directions and Challenges

We now present solution directions and corresponding challenges associated with each task in the above research goal.

1) *Cross-Domain TL*: We employ contrastive learning [43] to guide the target model f_θ^T in referencing the source model f^S , ensuring that target domain positive pairs (samples with the same label) yield similar representations, while negative pairs (samples with different labels) produce distinct representations. This enables f_θ^T to learn the underlying representations of samples from different classes with limited labeled data.

Challenge ①: Effectively selecting positive and negative pairs and similarity metrics for inducing the target model to learn robustness-relevant representations. In contrast to normal representations, which capture clean patterns that help the model distinguish between classes but may be highly sensitive to small perturbations, **robustness-relevant representations** refer to the encoded features that remain stable under adversarial perturbations, noise, or distribution shifts [44]. Appropriate and diverse contrastive pairs, including positive and negative pairs, help the model capture the correct relationships among the anchor sample, positive sample, and negative sample, promoting the learning of discriminative representations. However, contrastive pairs that emphasize representations related to adversarial and natural robustness in NID are underexplored.

Challenge ②: Transforming target domain samples to fit the input layer of the source model when the target and source domains differ in the number of input feature dimensions. Since the source model is pre-trained, it is necessary to adapt the target domain input samples to the source model by upscaling or downscaling the dimensions when the input feature space changes. Existing robustness-preserving TL (e.g., AFT [21], [35]) achieves this by resizing the resolution of images. Yet, such scaling is not suitable for network traffic and can lead to feature loss or distortion. Thus, an adaptive input feature transformation mechanism is needed to align the input feature dimensions between the target and source domains.

2) *Cross-Model TL*: We use multi-layer distillation, combining both logit-based and feature-based KD, with hidden representations and soft labels as knowledge carriers, to transfer robustness across model architectures within the same data domain and label space. Compared to existing robustness-preserving TL approaches (e.g., AD [20], [22]), which align only output distributions, our strategy allows the target model to also learn

intermediate abstract information from the source model, specifically robustness-relevant representations, thus mimicking the source model's reasoning processes.

Challenge ③: Efficiently extracting and aligning hidden representations between source and target models that may come from different hidden representation spaces. When the source and target models have different architectures, their representations at hidden layers may differ in dimensionality and abstract meaning. Thus, representation alignment and measurement methods are needed for transferring robustness-relevant representations from the source model's hidden space.

3) *Cross-Domain-and-Model TL*: We integrate contrastive learning and adaptive input dimension alignment mechanisms into hidden representation distillation to enable TL across both model architectures and data domains. Since the cross-domain TL tasks we consider also involve changes in the label space, the logit-based distillation, which relies on aligning soft labels, will be inactive in cross-domain-and-model TL scenarios.

Challenge ④: Balancing the clean performance and robustness of the target model on the target domain while mitigating the potential misguidance of the source model in the target domain. The robustness and clean performance of the target model are influenced not only by the TL algorithm but also by the inherent generalization ability of the source model's robustness and clean performance. A robust source foundation model—trained adversarially on a vast dataset—is likely to capture underlying and general data patterns. However, when the target model leverages the source model's representations or predictions for TL, the source model may also provide unreliable supervision for the target task in cross-domain TL scenarios. Thus, beyond adapting knowledge from the source model, the target model's robustness and clean performance must be further refined using the limited target domain data.

V. DESIGN OF CARD

This section presents the design of the proposed robustness-preserving transfer learning method, Contrastive Adversarial Representation Distillation (CARD), which aims to adapt the robustness and clean performance of a *robust* source model f^S to a target task across data domains and model architectures in a data-scarce setting. The core idea of CARD is to contrastively distill the hidden representations of a robust source model, especially the robustness-relevant representations, thereby producing a robust and accurate target model f_θ^T using limited labeled samples. Each sample in the target domain training dataset D^T serves as an **anchor**. f_θ^T learns hidden representations by comparing the anchor sample x with positive samples x^+ (transformed samples semantically similar to x) and negative samples x^- (samples in D^T dissimilar to x), to bring positive samples closer while pushing negative samples to be more orthogonal in the hidden representation space. Regarding the robustness-relevant representations, the hidden representations of f^S on adversarial and corrupted variants of x are adopted as the robustness carriers.

A. Architecture of CARD

The Dual Robustness-aware Contrastive View Construction and Dual Adaptive Dimension Alignment are key components for achieving robustness transfer and adaptation across different but related data domains and model architectures. The complete architecture of CARD is shown in Fig. 3.

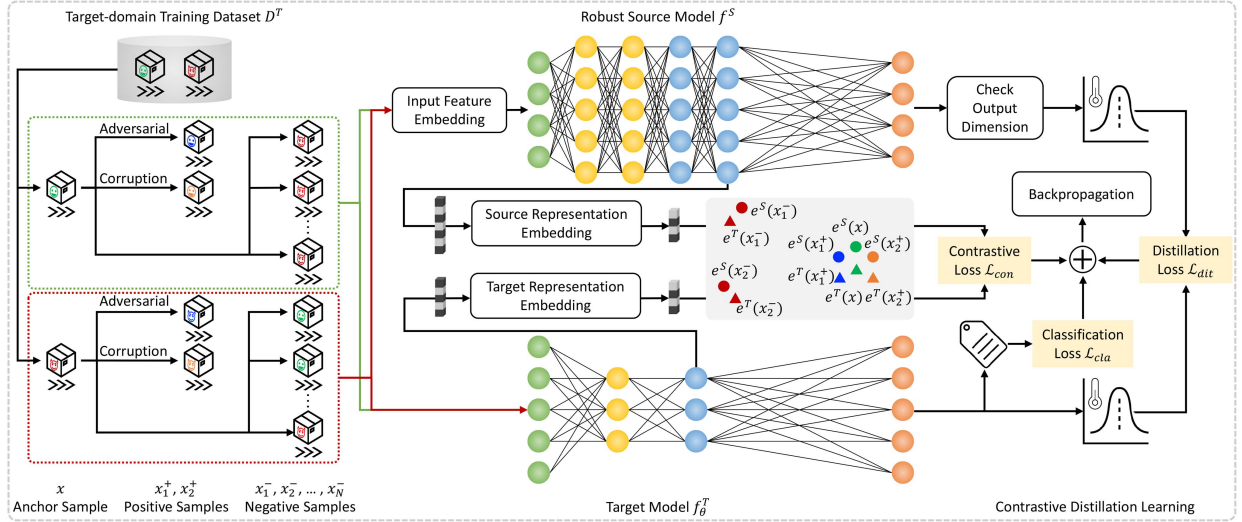


Fig. 3. Architecture of CARD. (1) Left: contrastive view construction with green for clean benign samples, red for clean malicious samples (different reds for different categories of anomalies), blue for naturally corrupted examples. (2) Middle: dual adaptive dimension alignment for input features and hidden representations. (3) Right: learning of target model based on TL losses ($\mathcal{L}_{con}$, $\mathcal{L}_{dit}$) and local classification loss ($\mathcal{L}_{cla}$).

1) *Dual Robustness-Aware Contrastive View Construction*: Contrastive pairs in contrastive learning (CL) [43] consist of **positive pairs** (x, x^+) , which include instances with similar features, and **negative pairs** (x, x^-) , which comprise dissimilar instances. The target model f_θ^T is trained to minimize the distance between samples in positive pairs while maximizing the distance between samples in negative pairs. In CARD, contrastive view construction enhances the robustness awareness.

a) *Positive Pairs*: Unlike standard CL [43], [45], where the single positive view x^+ is generated by standard augmentation (e.g., rotation, scaling, cropping, or flipping) to the anchor x , we introduce two robustness-aware positive views for network traffic data: the adversarial manipulation view x^* and the natural corruption view $\tilde{x}$, tackling Challenge ①.

- *Adversarial Manipulation View*: f^S is known to be robust against evasion attacks in CARD. Treating the adversarially manipulated version of the anchor as a positive view aims to extract hidden representations of f^S that are less sensitive to adversarial perturbations, thereby encouraging f_θ^T to learn **adversarial robustness-relevant representations**. We construct a positive pair $(x, x_1^+) = (x, x^*)$ based on the adversarial manipulation view x^* , which is crafted as (15).

$$x_1^+ = x^* = x + \delta^* = x + \arg \max_{\|\delta\|_p < \epsilon} \mathcal{L}(f_\theta^T(x + \delta), y), \quad (15)$$

where y is the truth label of $x \in D^T$. Moreover, we align the generation of δ with the threat model used in the robust training of f^S , ensuring consistency in perturbation budget ϵ , l_p norm type, and other factors, so that f_θ^T can adaptively inherit the robustness of f^S as its threat model evolves.

- *Natural Corruption View*: Since f^S is trained on a large and diverse dataset, it generally poses better generalization. Using the naturally corrupted version of the anchor as another positive view helps extract hidden representations of f^S that remain stable under distribution shifts, thereby encouraging f_θ^T to learn **natural robustness-relevant representations**. Thus, we construct a positive pair $(x, x_2^+) = (x, \tilde{x})$ based on the natural corruption view $\tilde{x}$, which is

generated as (16).

$$x_2^+ = \tilde{x} = x + \zeta = [x_t + \zeta_t, x_q + \zeta_q, x_l + \zeta_l], \quad (16)$$

where the time-related, quantity-related, and length-related network traffic feature dimensions (x_t, x_q, x_l) of $x \in D^T$ are distorted by the random noise vectors $\zeta = (\zeta_t, \zeta_q, \zeta_l)$ sampled from the Gaussian distribution $\mathcal{N}(0, 1)$.

b) *Negative Pairs*: Negative samples x^- provide more diverse contrastive signals than positive samples x^+ to help the model learn discriminative representations for class separation. In standard CL, each anchor x usually has only one or a few x^+ , whereas x^- are more abundant and often drawn from the same mini-batch as x (e.g., SimCLR [43]) or from a dynamically updated **memory bank** (a.k.a. memory buffer or queue) that stores previously seen x^- (e.g., MoCo [46]).

To enable CARD in a resource-limited TL environment, we adopt a memory bank-based strategy to manage x^- , eliminating reliance on large batch sizes and reducing computational and memory overhead in CL. In addition, considering the multi-class classification scenario in NID, the target model f_θ^T is required to learn not only the coarse-grained distinction between benign and malicious traffic patterns, but also the fine-grained differences among various malicious traffic categories. Specifically, we propose the following strategy for selecting negative pairs $\{(x, x_1^-), \dots, (x, x_N^-)\}$ for the anchor $x \in D^T$.

- When x is a benign sample, we select N malicious samples with different intrusion types from the memory bank as x^- . Particularly, we prioritize intrusion samples that exhibit significant differences from x in features such as communication protocols, network service, port number, number of connection attempts, flow size, etc.
- When x is a malicious sample, we select N samples from the memory bank as x^- , including benign traffic samples and malicious traffic samples with different classes from x .

It is worth noting that we do not use adversarial or naturally corrupted versions of selected x^- to augment negative pairs. (i) Each sample x in D^T will eventually serve as an anchor and be compared with its robustness-aware positive views. Thus,

when it acts as a negative sample, we hope to prevent f_θ^T from repeatedly learning its perturbation pattern, thereby preventing overfitting to the perturbation and losing generalization on clean samples. (ii) In addition, past work shows that expanding x^- has diminishing returns [47] and poses a collision-coverage risk, meaning the model may overfit to redundant patterns when negative samples are too similar to each other [48]. (iii) Also, generating adversarial examples for extensive x^- is computationally expensive. Thus, we use only clean samples as x^- to balance performance and cost.

Subsequently, positive and negative pairs will be simultaneously fed into f^S and f_θ^T for contrastive transfer learning.

2) *Dual Adaptive Dimension Alignment*: First, to support TL across data domains, particularly those with different input feature spaces, we use an Input Feature Dimension Alignment strategy when the number of feature dimensions d^S of the source domain samples is inconsistent with d^T of the target domain samples. Second, to support TL across model architectures, especially across different hidden representation spaces, we use a Hidden Representation Dimension Alignment strategy to align the numbers of extracted hidden representation dimensions d_e^S from f^S and d_e^T from f_θ^T .

a) *Input Feature Dimension Alignment*: When learning f_θ^T from the pre-trained f^S , we need to input $x \in D^T$ into f^S to obtain contrastive distillation signals. However, when d^T differs from d^S , f^S cannot directly process x as input. To address Challenge ②, we employ the following input feature dimension alignment strategy. We align the d^T -dimensional $x \in D^T$ with the dimension d^S of the source domain input feature space using a learnable **Input Feature Embedding** module $E_{\theta I}$. It consists of a linear transformation layer $h_{\theta I}(x)$ with optimizable parameters θ^I and an l_2 normalization function $m(h_{\theta I}(x)) = h_{\theta I}(x)/\|h_{\theta I}(x)\|_2$. The two hyperparameters of $h_{\theta I}(x)$, input dimension and output dimension, are automatically set to d^T and d^S , respectively. This approach allows CARD to adaptively align input feature dimensions for cross-domain TL. Since the performance of $E_{\theta I}$ influences the hidden representation and output probability distribution of f^S , thereby impacting both contrastive and distillation losses, we jointly optimize $E_{\theta I}$ alongside f_θ^T .

b) *Hidden Representation Dimension Alignment*: A hidden representation e refers to the intermediate feature representation learned by a neural network at a specific hidden layer. In CARD, we calculate the contrastive loss based on the hidden representations, e^S and e^T , from the last hidden layers of f^S and f_θ^T , respectively. To enable TL across model architectures, particularly those with mismatched representation dimensions, we use two learnable **Hidden Representation Embedding** modules, namely the Source Hidden Representation Embedding $E_{\theta S}$ and the Target Hidden Representation Embedding $E_{\theta T}$, to map the raw e^S and e^T , which may have different dimensions, into a shared space of dimension d_e , solving Challenge ③. These embedding modules structurally resemble the Input Feature Embedding module, but differ in the input and output dimensions of the linear transformation layers. Since the performance of $E_{\theta S}$ and $E_{\theta T}$ directly impacts the contrastive loss, they are also jointly optimized along with f_θ^T . Afterward, the aligned hidden representations are used for CL-based representation distillation, while the original hidden representations are fed into the output layers for output probability distribution-based distillation and adversarial classification.

B. Learning Objective of CARD

Algorithm 1 outlines the transfer learning process of the target model f_θ^T from the robust source model f^S using limited labeled data in the target domain training dataset D^T .

We adopt a comprehensive loss, $\mathcal{L}_{card}$, composed of three terms: Representation Contrastive Loss $\mathcal{L}_{con}$, Output Distillation Loss $\mathcal{L}_{dit}$, and Adversarial Classification Loss $\mathcal{L}_{cla}$. Our goal is to jointly optimize f_θ^T , the Input Feature Embedding module $E_{\theta I}$, and the Hidden Representation Embedding modules $E_{\theta S}$ and $E_{\theta T}$ by minimizing $\mathcal{L}_{card}$, as defined in (17).

$$\min_{\theta, \theta^I, \theta^S, \theta^T} \mathcal{L}_{card} = \alpha \cdot \mathcal{L}_{con} + \beta \cdot \mathcal{L}_{dit} + \gamma \cdot \mathcal{L}_{cla}, \quad (17)$$

where $\alpha, \beta, \gamma \in [0, 1]$. Section IX shows the ablation study.

1) *Representation Contrastive Loss*: To transfer robustness-relevant hidden representations from f^S to f_θ^T that remain stable under adversarial perturbations and natural corruptions, we define the contrastive loss $\mathcal{L}_{con}$ by extending CRD [47] to include two robustness-aware positive views. $\mathcal{L}_{con}$ consists of two terms: (i) $\mathcal{L}_{con}^T$ —the contrastive loss between representations (e^S, e^T) of the anchor x from f^S and f_θ^T , as well as representations (e^{+T}, e^{-T}) of the positive and negative samples (x^+, x^-) from f_θ^T ; and (ii) $\mathcal{L}_{con}^S$ —the contrastive loss between representations (e^S, e^T) as well as representations (e^{+S}, e^{-S}) of (x^+, x^-) from f^S , as defined in (18).

$$\mathcal{L}_{con} = \mathcal{L}_{con}^T(e^S, e^T, e^{+T}, e^{-T}) + \mathcal{L}_{con}^S(e^T, e^S, e^{+S}, e^{-S}) \quad (18)$$

where e denotes the embedded hidden representations after dimensional alignment. Specifically, $\mathcal{L}_{con}^T$ and $\mathcal{L}_{con}^S$ are calculated according to (19) and (20), respectively, where e^S and e^T are successively regarded as the anchor representation.

$$\begin{aligned} \mathcal{L}_{con}^T(e^S, e^T, \{e_j^{+T}\}_{j=1}^2, \{e_j^{-T}\}_{j=1}^N) &= -[\log h(e^S, e^T) \\ &+ \sum_{j=1}^2 \log h(e^S, e_j^{+T}) + \sum_{j=1}^N \log(1 - h(e^S, e_j^{-T}))], \end{aligned} \quad (19)$$

$$\begin{aligned} \mathcal{L}_{con}^S(e^T, e^S, \{e_j^{+S}\}_{j=1}^2, \{e_j^{-S}\}_{j=1}^N) &= -[\log h(e^T, e^S) \\ &+ \sum_{j=1}^2 \log h(e^T, e_j^{+S}) + \sum_{j=1}^N \log(1 - h(e^T, e_j^{-S}))]. \end{aligned} \quad (20)$$

N denotes the number of negative samples, e_j^+ and e_j^- denote the embedded representations of the j th positive sample x_j^+ and the j th negative sample x_j^- , respectively. The contrastive score of any contrastive pair (a, b) is calculated as in (21) [47], with higher scores indicating greater similarity.

$$h(a, b) = \frac{\text{score}(a, b)}{\text{score}(a, b) + N \cdot P_n} = \frac{\exp(a \cdot b / \tau)}{\exp(a \cdot b / \tau) + N / M}, \quad (21)$$

where M is the training dataset D^T size, P_n is the noise probability in negative samples, τ is the temperature parameter.

2) *Output Distillation Loss*: To guide f_θ^T in imitating the output probability distributions, o^S and o_{adv}^S , from f^S on both clean input x and adversarial input $x_1^+ = x^* = x + \delta^*$, we incorporate a logit-based distillation loss, as defined in (22).

$$\begin{aligned} \mathcal{L}_{dit}(o^T, o^S, o_{adv}^T, o_{adv}^S) \\ = \text{KL}(o^{S^\tau} | o^{T^\tau}) + \text{KL}(o^{S^\tau} | o_{adv}^{T^\tau}) + \text{KL}(o_{adv}^{S^\tau} | o_{adv}^{T^\tau}) \end{aligned}$$

Algorithm 1: Contrastive Adversarial Representation Distillation.

Input: d^T -dimensional target domain training sample $x \in D^T$, pretrained robust source model f^S with input dimension d^S and output dimension c^S , contrastive representation dimension d_e , input feature embedding module E_{θ^I} , hidden representation embedding modules E_{θ^S} and E_{θ^T} , adversarial example generator G_{adv} , corrupted example generator G_{cor} .

Output: robust target model f_{θ}^T with output dimension c^T .

```

1: for  $i = 1$  to  $\text{len}(D^T)$  do
2:   select anchor sample  $x \leftarrow x_i \in D^T$ 
3:   generate adversarial example  $x + \delta \leftarrow G_{adv}(x, f_{\theta}^T)$ 
4:   generate corrupted example  $x + \zeta \leftarrow G_{cor}(x)$ 
5:   select  $N$  samples  $x_j^-$  from  $D^T$  with label  $\neq y$ 
6:   construct positive pairs
    $X_p = (x, x_1^+, x_2^+) \leftarrow (x, x + \delta, x + \zeta)$ 
7:   construct negative pairs  $X_n = (x, x_1^-, \dots, x_N^-)$ 
8:   if  $d^T \neq d^S$  then
9:      $\hat{X}_p, \hat{X}_n \leftarrow \text{apply InpEmbedding } E_{\theta^I}(d^T, d^S)$ 
       on  $X_p, X_n$ 
10:     $X_p \leftarrow \hat{X}_p, X_n \leftarrow \hat{X}_n$ 
11:   end if
12:   feed anchor  $x$  to source model  $f^S$  and target model  $f_{\theta}^T$ 
13:   extract hidden representation
    $e^S \leftarrow f^{r^S}(x), e^T \leftarrow f_{\theta}^{r^T}(x)$ 
14:    $e^S \leftarrow e^{S^E} \leftarrow \text{apply RepEmbedding } E_{\theta^S}(e^S, d_e)$ 
15:    $e^T \leftarrow e^{T^E} \leftarrow \text{apply RepEmbedding } E_{\theta^T}(e^T, d_e)$ 
16:   output logit value  $o^S \leftarrow f^S(x), o^T \leftarrow f_{\theta}^T(x)$ 
17:   for  $j = 1$  to  $2$  do
18:     feed positive sample  $x_j^+$  to source and target models
19:     hidden representation
      $e_j^{+S} \leftarrow f_{\theta}^{r^S}(x_j^+), e_j^{+T} \leftarrow f_{\theta}^{r^T}(x_j^+)$ 
20:      $e_j^{+S} \leftarrow e_j^{+S^E} \leftarrow \text{apply RepEmbedding } E_{\theta^S}(e_j^{+S}, d_e)$ 
21:      $e_j^{+T} \leftarrow e_j^{+T^E} \leftarrow \text{apply RepEmbedding } E_{\theta^T}(e_j^{+T}, d_e)$ 
22:     if  $j = 1$  then
23:       output logits  $o_{adv}^S \leftarrow f^S(x_j^+), o_{adv}^T \leftarrow f_{\theta}^T(x_j^+)$ 
24:     end if
25:   end for
26:   for  $j = 1$  to  $N$  do
27:     feed negative sample  $x_j^-$  to source and target models
28:     hidden representation  $e_j^{-S} \leftarrow f_{\theta}^{r^S}(x_j^-),$ 
      $e_j^{-T} \leftarrow f_{\theta}^{r^T}(x_j^-)$ 
29:      $e_j^{-S} \leftarrow e_j^{-S^E} \leftarrow \text{apply RepEmbedding } E_{\theta^S}(e_j^{-S}, d_e)$ 
30:      $e_j^{-T} \leftarrow e_j^{-T^E} \leftarrow \text{apply RepEmbedding } E_{\theta^T}(e_j^{-T}, d_e)$ 
31:   end for
32:   compute contrastive loss
    $\mathcal{L}_{con}^T(e^S, e^T, e_j^{+T}, e_j^{-T})_{j=1}^N$ 

```

```

33:   compute contrastive loss
    $\mathcal{L}_{con}^S(e^T, e^S, e_j^{+S}, e_j^{-S})_{j=1}^N$ 
34:   compute contrastive loss  $\mathcal{L}_{con} = \mathcal{L}_{con}^T + \mathcal{L}_{con}^S$ 
35:   compute classification loss  $\mathcal{L}_{cla}(o^T, o_{adv}^T, y)$ 
36:   if  $c^S = c^T$  then
37:     compute distillation loss
      $\mathcal{L}_{dit}(o^T, o^S, o_{adv}^T, o_{adv}^S)$ 
38:     compute weighted loss
      $\mathcal{L}_{card} = \alpha \cdot \mathcal{L}_{con} + \beta \cdot \mathcal{L}_{dit} + \gamma \cdot \mathcal{L}_{cla}$ 
39:   else
40:     compute weighted loss
      $\mathcal{L}_{card} = \alpha \cdot \mathcal{L}_{con} + \gamma \cdot \mathcal{L}_{cla}$ 
41:   end if
42:   update parameters of  $f_{\theta}^T, E_{\theta^S}, E_{\theta^T}, E_{\theta^I}$ 
43: end for

```

$$\begin{aligned}
&= \text{KL}(f^S(x)^\tau | f_{\theta}^T(x)^\tau) + \text{KL}(f_{\theta}^S(x)^\tau | f_{\theta}^T(x^*)^\tau) \\
&\quad + \text{KL}(f_{\theta}^S(x^*)^\tau | f_{\theta}^T(x^*)^\tau) \\
&= \text{KL}(S(f^S(x)/\tau) | S(f_{\theta}^T(x)/\tau)) \\
&\quad + \text{KL}(S(f^S(x)/\tau) | S(f_{\theta}^T(x^*)/\tau)) \\
&\quad + \text{KL}(S(f^S(x^*)/\tau) | S(f_{\theta}^T(x^*)/\tau)), \tag{22}
\end{aligned}$$

where o denotes the output probability distribution from the softmax layer S , τ is a temperature factor to scale the logits. We avoid enforcing output dimension alignment to prevent the prediction errors of f^S on D^T from misleading f_{θ}^T in cross-domain TL involving different label spaces. Thus, we apply $\mathcal{L}_{dit}$ only when the label spaces of f^S and f_{θ}^T are consistent.

3) *Adversarial Classification Loss:* To balance clean performance and robustness of f_{θ}^T in TL, we use a CrossEntropy-based classification loss defined in (23), integrating standard and adversarial training. In a data scarcity setting, weighted $\mathcal{L}_{cla}$ enables f^T to learn clean example x and adversarial example x^* with truth label y from its own domain, rather than relying solely on possibly imprecise signals from f^S , mitigating Challenge 4 without incurring excessive costs.

$$\begin{aligned}
\mathcal{L}_{cla}(o^T, y) &= \mathcal{L}_{CE}(o^T, y) + \mathcal{L}_{CE}(o_{adv}^T, y) \\
&= \mathcal{L}_{CE}(f_{\theta}^T(x), y) + \mathcal{L}_{CE}(f_{\theta}^T(x^*), y). \tag{23}
\end{aligned}$$

VI. EVALUATION SETUP

A. Testbed

We implemented CARD in PyTorch 2.2.0 and Python 3.9.18. Each experiment ran three times with seeds {41, 42, 43} on an NVIDIA GeForce RTX 3090 using Torch Cuda 12.1¹.

B. Baseline Methods

The baselines for comparison include NID-specific standard transfer learning (TL), such as fine-tuning and distillation, and SOTA robustness-preserving TL originally designed for images, like adversarial fine-tuning and distillation (see Table I).

¹The code is available at <https://github.com/RobTransfer/CARD>.

TABLE I
FUNCTIONAL COMPARISON OF ROBUSTNESS-PRESERVING TL BASELINES

TL Method	NID-Specific	Robust TL	Binary -Class	Multi-Class	Allowed Data Domains	Source-Target Architectures	Difference Domains & Models
FT [17]	●	○	●	○	●	○	○
KD [18]	●	○	●	○	○	●	○
FRFE [6]	○	●	○	●	●	○	○
TWINS [21]	○	●	○	●	●	○	○
VAD [20]	○	●	○	●	○	●	○
AAD [22]	○	●	○	●	○	●	○
CARD	●	●	●	●	●	●	●

TABLE II
SOURCE DOMAIN AND TARGET DOMAIN DATASETS INFORMATION

Input Feature Space	Domain	Training Dataset	Class Num	Usefulness	Used Data Ratio	Used Data Number	Used Benign Data	Used Malicious Data
Same within TL	Source	UNSW-NB15- <i>NoExploit</i>	9	Source AdvTrain	100%	86440	51900	34540
	Target	UNSW-NB15- <i>WithExploit</i>	2	TL across Domain	5%	4322	2595	1727
Different within TL	Source	NSL-KDD- <i>All</i>	5	Source AdvTrain	100%	125980	67360	58620
	Target	UNSW-NB15- <i>All</i>	10	TL across Domain	5%	5299	2595	2704

C. Datasets

We conducted evaluations on two source-target domain pairs (see Table II). The widely used NSL-KDD [49] benchmark (41 features) mitigates redundancy, while UNSW-NB15 [50] (49 features) includes evolved attacks from real-world networks.

1) *TL Across Domains With the Same Input Feature Space*: It represents a TL scenario in NID where the target dataset is created from networks using the same protocol and collection tools as the source dataset, but with new attack types.

- UNSW-NB15-NoExploit is the source dataset D^S from UNSW-NB15, including *Benign* and 8 malicious attack classes (*Fuzzers*, *Analysis*, *Backdoors*, *DoS*, *Generic*, *Reconnaissance*, *Shellcode*, and *Worms*), excluding *Exploits*.
- UNSW-NB15-WithExploit is the target dataset D^T with *Benign* and *Exploits* intrusion classes from UNSW-NB15.

2) *TL Across Domains With Different Input Feature Spaces*: It represents the TL scenario in NID where the target dataset is created from different networks with distinct protocols, varied feature processing tools, and changed intrusion distributions.

- NSL-KDD-All is the source dataset D^S from NSL-KDD, including *Benign* and 4 major malicious attack classes (*Dos*, *Probe*, *R2L*, and *U2R*) consisting of complete 22 specific intrusion types.
- UNSW-NB15-All is the target dataset D^T from UNSW-NB15, including *Benign* and complete 9 intrusion classes.

D. Classification Models

We evaluated two source-target model architecture pairs (see Table III). The WideResNet-34-10 [51], pre-trained using PGD-AT [23] as source dataset, was used as the robust source model architecture, with the lightweight and efficient ResNet-18 [52] and MobileNet [53] as the target models, following the settings in robustness-preserving TL baselines [6], [20], [21], [22].

TABLE III
MODEL ARCHITECTURE AND CAPACITY INFORMATION

Model Role	Model Architecture	Building Block	Total Params	Forward Size	Params Size	Total Size
Source	WideResNet-34-10	Residual	46,159,545	7.38M	176.08M	183.47M
Target	ResNet-18	Residual	11,172,297	1.29M	42.62M	43.91M
Target	MobileNet	Separable	3,215,625	1.69M	12.27M	13.95M

TABLE IV
CORRELATION BETWEEN TARGET AND SOURCE TASKS IN EVALUATION

Section	TL Scenarios	Label Space	Feature Space	Building Block
VII-A	TL across	Different	Same	Same
VIII-A	Data Domains		Different	Same
VII-B	TL across	Same	Same	Similar
VIII-B	Model Architectures		Different	Different
VII-C	TL across both Domain&Model	Different	Same	Similar
VIII-C			Different	Different

1) *TL Across Models With Similar Building Blocks*: It simulates TL tasks in NID that transfer knowledge to a compressed model with similar but smaller blocks than the source model.

- WideResNet-34-10, as the source model, is built on wide residual blocks with 34 layers and convolutional channels 10 times wider than standard ResNet, offering improved performance at the cost of increased computational load.
- ResNet-18, as the target model, is built on basic residual blocks with 18 layers and normal convolutional channels.

2) *TL Across Models With Different Building Blocks*: It simulates challenging TL tasks in NID, where the resource-constrained target model uses much different, lighter blocks.

- WideResNet-34-10 is the source model architecture.
- MobileNet is the target model built on depthwise separable convolutional blocks with fewer parameters and computations. It is suited for mobile or embedded devices in low-resource environments at the cost of performance.

E. Testing Pipeline

We tested CARD and baselines in TL scenarios outlined in Fig. 2 with varying model architectures and data domains. In particular, we evaluated them in two settings: TL using (5%) **scarce target domain training data** (Section VII), and (5%-50%) **expanded target domain training data** (Section VIII). Furthermore, we assessed two detection granularities for each TL task: multi-class classification and binary classification. Table IV shows the differences between the source and target tasks involved in the evaluation of different TL scenarios.

F. Hyperparameter Selection

1) *Adversarial Pretraining*: While adversarially pre-trained models on large-scale images have been released and used in robustness-preserving TL baselines [6], [20], [21], [22], none exist for network traffic. Thus, we trained robust source NID models using adversarial training (as defined in Section III-C) settings from baselines. The perturbation δ was generated using l_∞ -PGD with 20 iterations, an ϵ of 0.1, and a step size of 0.05.

2) *Transfer Learning*: All target models were trained with a batch size of 1024 for 50 epochs with an initial learning rate of 0.1, incorporating an early stopping mechanism. The temperature τ in all baselines was set to 0.1. The perturbation settings involved in adversarial fine-tuning and distillation as

well as CARD training are the same as in pretraining. For CARD, the coefficients were set as follows: $\alpha = 0.33$ for $\mathcal{L}_{con}$, $\beta = 0.33$ for $\mathcal{L}_{dit}$, and $\gamma = 0.5$ for $\mathcal{L}_{cla}$. This ensures equal contributions from $\mathcal{L}_{con}$ and $\mathcal{L}_{dit}$ while placing sufficient emphasis on $\mathcal{L}_{cla}$, which is supervised by the ground-truth labels of target domain samples. The number of negative samples, N , was set to 4,096, following the recommended settings in CRD [47] to balance the diminishing return in CL.

3) *Attack Configuration*: Our evaluation of adversarial evasion attacks consists of three parts: (i) l_∞ -PGD [23] (assessed in Sections VII, VIII, and IX), which is widely used and maintains the same perturbation settings as in adversarial pretraining; (ii) AutoAttack (assessed in Section X), combining DeepFool [54] and Auto-PGD (APGD) [55], with iterations number increased to 100 to explore attack strength; and (iii) our newly designed adaptive attacks (detailed in Section X) for cross-domain, cross-model, and cross-domain-and-model TL tasks.

G. Evaluation Metrics

We evaluated the performance of the target model on different versions of the target domain test samples using the following metrics: Accuracy (Acc), F1-score (F1), Recall, Precision (Precis), False Positive Rate (FPR), and False Negative Rate (FNR). Results on clean test samples (e.g CleAcc), adversarial test samples (e.g AdvAcc), and corrupted test samples (e.g CorAcc) reveal the clean performance, adversarial robustness, and natural robustness of the target model, respectively. Each reported value represents the average result from three seeds.

VII. EVALUATION IN DATA-SCARCE SCENARIOS

We experimentally compared CARD and SOTA adversarial fine-tuning and distillation using 5% of target domain training data in three TL scenarios: across data domains (Section VII-A), across model architectures (Section VII-B), and across both domains and models (Section VII-C).

A. Transfer Learning Across Data Domains

We compare CARD with four baselines: training from scratch (StdTrain), standard fine-tuning (FT [17]), and SOTA adversarial fine-tuning methods (TWINS [21] and FRFE [6]).

1) *TL Across Domains With the Same Input Feature Space*: See Figs. 4(a) and 5(a) for detailed results.

Results: ① Compared to StdTrain, in the multi-class case, CARD boosts AdvAcc and CorAcc by 51.14% and 46.33%, while CleAcc and CleF1 rise by 10.63% and 14.25%. In the binary case, CARD achieves improvements on AdvAcc (50.14%), CorAcc (36.87%), CleAcc (9.96%), and CleF1 (14.81%). ② Compared to FT, in multi-class case, CARD improves AdvAcc by 42.06%, CorAcc by 4.97%, and CleAcc by 4.36%, while AdvF1 rises by 38.33% and CleF1 by 14.15%. In binary case, AdvAcc improves by 42.20%, CorAcc by 22.10%, and CleAcc by 3.32%. This answers research question ① defined in Section I, showing that standard FT-based TL from a robust source model rarely ensures target domain robustness. ③ Compared to AFT methods, in the multi-class case, CARD achieves 6.33% and 2.33% higher AdvAcc than TWINS and FRFE, and 3.67% and 4.33% higher CorAcc. In the binary case, CARD leads TWINS and FRFE by 3.33% and 30% in AdvAcc, and 5.33% and 9.67% in CorAcc, respectively. This addresses research question ②,

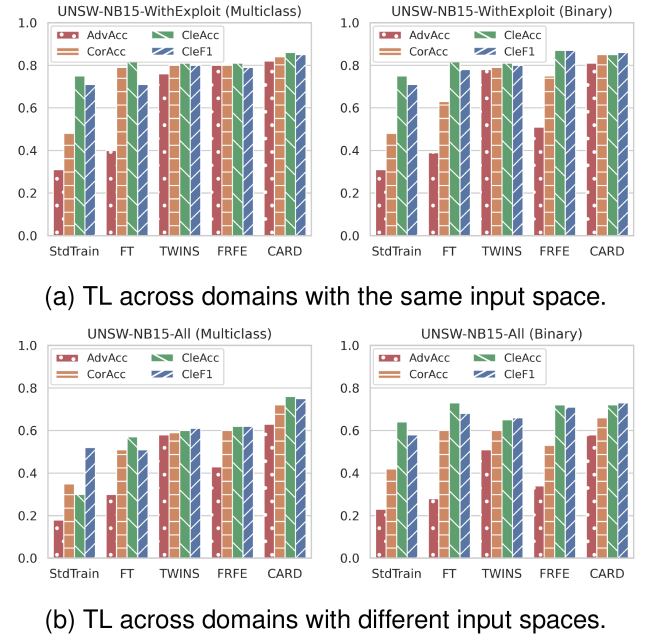


Fig. 4. Comparison with SOTA fine-tuning methods on cross-domain TL. (a) Generate UNSW-NB15-WithExploit detectors by fine-tuning robust UNSW-NB15-NoExploit detectors. (b) Generate UNSW-NB15-All detectors by fine-tuning robust NSL-KDD-All detectors.

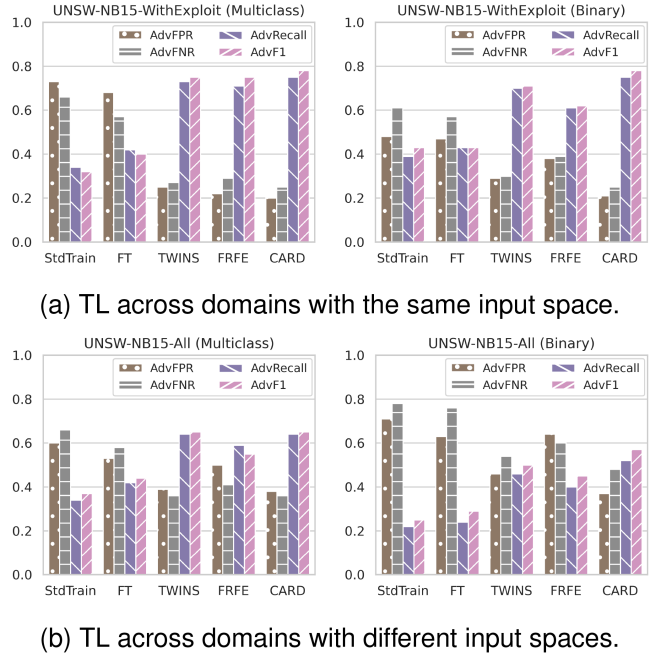


Fig. 5. Comparison of adversarial robustness of the target model against evasion attacks with SOTA fine-tuning methods on cross-domain TL using finer-grained adversarial metrics. The setup is the same as Fig. 4.

showing that CARD outperforms SOTA in transferring robustness from a robust source model to a new domain with limited data.

2) *TL Across Domains With Different Input Feature Spaces*: See Figs. 4(b) and 5(b) for detailed results.

Results: ① Compared to StdTrain, CARD increases AdvAcc and CorAcc by 39.19% and 37.52% in the multi-class case, and

shows improvement in binary case. ② Compared to FT, CARD boosts AdvAcc by 27.5% and 29.65% in the multi-class and binary cases, while CorAcc increases by 20.84% and 6.03%. In multi-class case, CleAcc, CleF1, AdvF1 increases by 18.76%, 23.97%, and 20.33%, respectively, validating our answer to question ①. ③ Compared to AFT methods, in multi-class case, CARD obtains equivalent AdvAcc to TWINS and surpasses FRFE, while CorAcc is 13.33% and 12% higher. In binary case, AdvAcc rises by 7.33% and 24%, and CorAcc by 5.33% and 13%. This further solves question ②.

B. Transfer Learning Across Model Architectures

We compare CARD with four baselines: training from scratch (StdTrain), standard distillation (KD [18]), and SOTA adversarial distillation methods (VAD [20] and AAD [22]).

1) *TL Across Models With Similar Building Blocks*: See Figs. 6(a) and 7(a) for detailed results.

Results: ① Compared to KD, AdvAcc and CorAcc increase by 37.81% and 18.00% in the multi-class case, and by 26.67% and 24.00% in binary case. For question ①, this suggests that a target model derived from a robust source model via standard KD-based TL does not inherit robustness. ② Compared to AD methods, CARD improves AdvAcc and CorAcc by 11.67% and 10.00% on average in multi-class case, and by 10.00% and 4.67% in binary case. For question ②, This shows CARD surpasses SOTA in robustness transfer across architectures.

2) *TL Across Models With Different Building Blocks*: See Figs. 6(b) and 7(b) for detailed results.

Results: ① Compared to KD, AdvAcc and CorAcc increase by 44.67% and 31.33% in the multi-class case, and by 47.33% and 24.00% in the binary case. ② Compared to AD methods, CARD boosts AdvAcc and CorAcc by 15.17% and 26.00% on average in the multi-class case, and by 12.00% and 8.33% in the binary case. This reinforces our validation of ① and ②.

C. Transfer Learning Across Both Domains and Models

Due to the lack of robustness-preserving TL methods across both domains and models, we use StdTrain as the baseline.

1) *TL Across Data Domains With the Same Input Feature Space*: See Figs. 8(a), (b) and 9(a), (b) for results.

Results: In the multi-class case, CARD achieves AdvAcc and CorAcc of 81% and 84% on ResNet-18, surpassing baseline by significant margins. On MobileNet, although the advantage slightly decreases due to the larger structural difference, CARD still outperforms the baseline by 41.45% and 36.42% in AdvAcc and CorAcc. In addition, CARD surpasses StdTrain in CleAcc, CleF1, CleRecall, and ClePrecis while lowering CleFPR and CleFNR.

2) *TL Across Data Domains With Different Input Feature Spaces*: See Figs. 8(c), (d) and 9(c), (d) for results.

Results: The target model's performance declines compared to TL within the same input feature space due to NSL-KDD-All having a higher-dimensional label space than UNSW-NB15-WithExploit, which increases TL complexity. Nevertheless, CARD outperforms StdTrain on robustness metrics with lower standard deviations. AdvAcc and CorAcc reach 60% and 61% on ResNet-18, and 60% and 54% on MobileNet in the multi-class case, significantly surpassing StdTrain. Moreover, CARD effectively balances clean performance, reducing CleFNR and

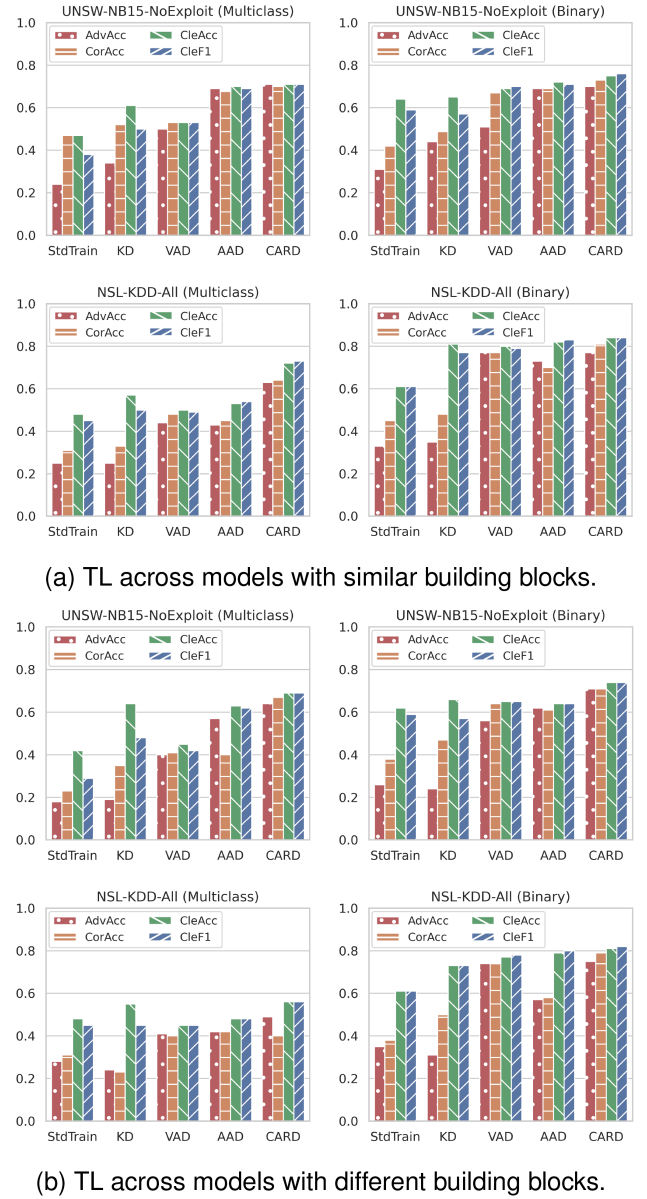


Fig. 6. Comparison with SOTA distillation methods on cross-model TL. (a) Distill robust WideResNet-34-10-based UNSW-NB15-NoExploit detectors to ResNet-18 and MobileNet. (b) Distill robust WideResNet-34-10-based NSL-KDD-All detectors to ResNet-18 and MobileNet.

CleFPR by 28.2% and 26.92% on ResNet-18, and by 28.87% and 29.39% on MobileNet in multi-class detection.

Summary: CARD surpasses baselines (including StdTrain, standard FT and KD, as well as SOTA adversarial FT and distillation methods) in transferring robustness and maintaining clean performance to different data domains and lightweight NID models with scarce (5%) target domain training data.

VIII. EVALUATION IN DATA-EXPANDING SCENARIOS

We incrementally increased the target dataset size to 5%, 10%, 20%, 30%, 40%, 50% to expand the available target domain training data, and compared the performance changes of CARD and SOTA adversarial fine-tuning and distillation in three TL scenarios: across data domains (Section VIII-A), across model

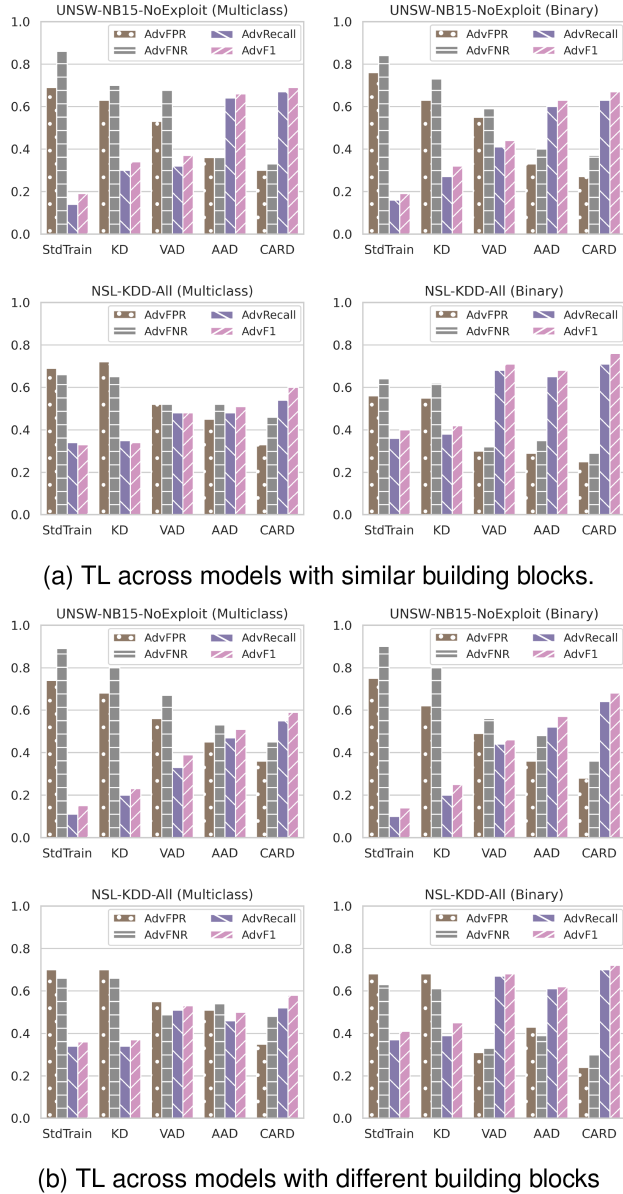


Fig. 7. Comparison of adversarial robustness of the target model against evasion attacks with SOTA distillation methods on cross-model architecture TL using finer-grained metrics. The experimental setup is the same as Fig. 6.

architectures (Section VIII-B), and across both domains and models (Section VIII-C).

A. Transfer Learning Across Data Domains

We compare CARD and baselines in a challenging cross-domain TL scenario with different input feature spaces for multi-class detection. See Fig. 10 for detailed results.

Results: ① CARD exhibits a more stable and gradual variation than all baselines, whereas baselines experience significant gains due to data expanding. This addresses research question **iii** defined in Section I, showing that CARD generalizes best with limited data and avoids over-reliance on expanded data. ② Although data expansion narrows clean performance gaps, CARD continuously outperforms all baselines. Moreover, the advantages of CARD in adversarial robustness (AdvAcc) and

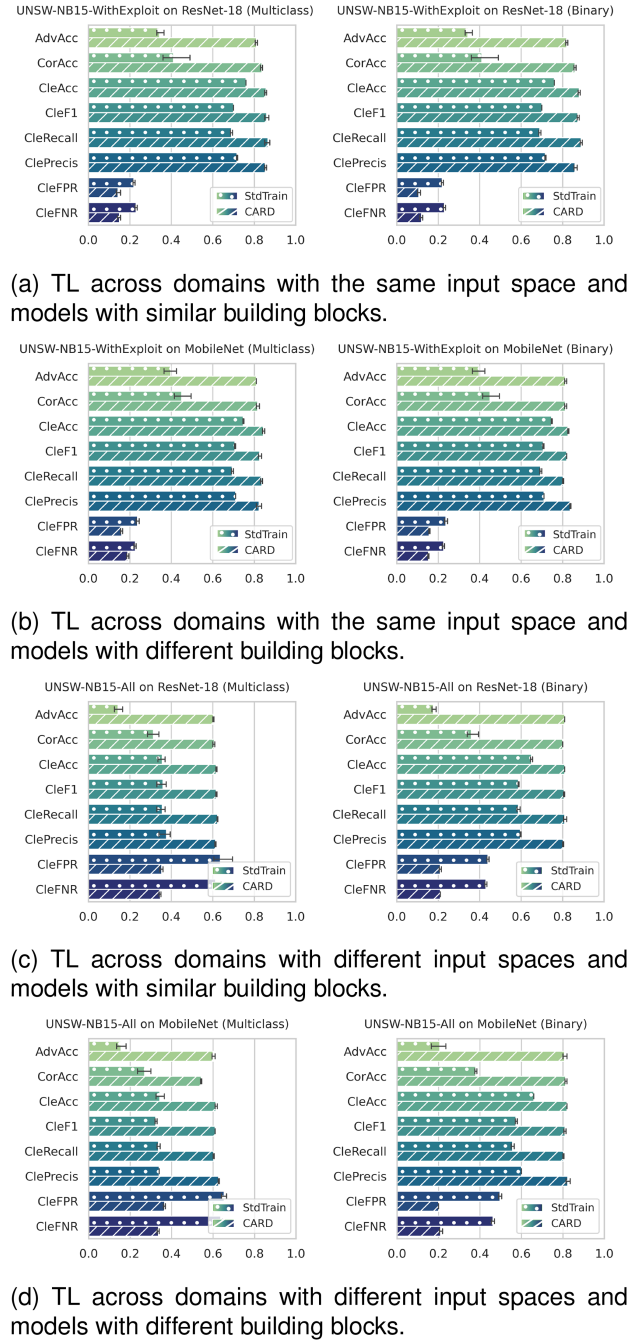


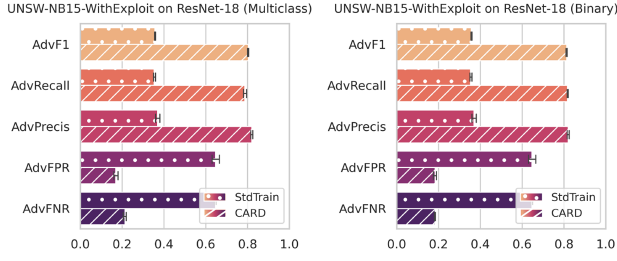
Fig. 8. Comparison with training from scratch on cross-domain & model TL. (a) Generate ResNet-18 and MobileNet UNSW-NB15-WithExploit detectors from robust UNSW-NB15-NoExploit detectors. (b) Generate ResNet-18 and MobileNet UNSW-NB15-All detectors from NSL-KDD-All detectors.

natural robustness (CorAcc) persist, confirming its superiority in both robustness and clean performance as data grows.

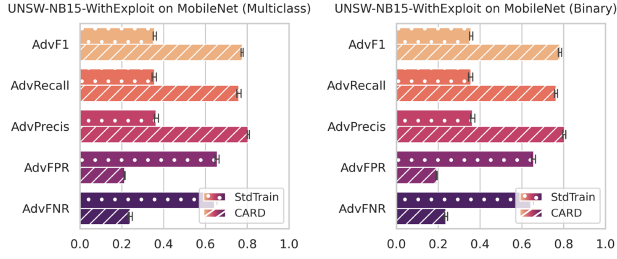
B. Transfer Learning Across Model Architectures

We compared CARD and baselines in a challenging cross-model TL scenario where a multi-class detector is compressed into a smaller one with different building blocks (see Fig. 11).

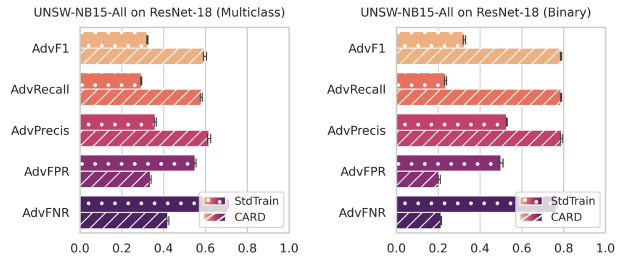
Results: ① Throughout the data expansion, CARD consistently outperforms baselines across all metrics, particularly on



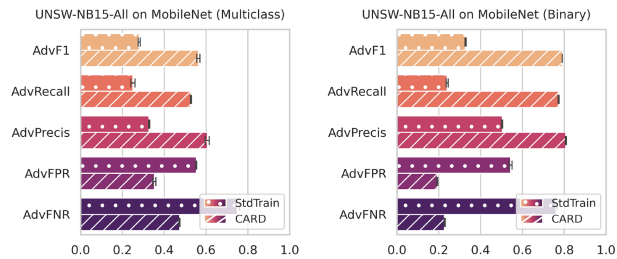
(a) TL across domains with the same input space and models with similar building blocks.



(b) TL across domains with the same input space and models with different building blocks.



(c) TL across domains with different input spaces and models with similar building blocks.



(d) TL across domains with different input spaces and models with different building blocks.

Fig. 9. Comparison of adversarial robustness of the target model against evasion attacks with training from scratch (StdTrain) on cross-domain & model TL Tasks. The experimental setup is the same as Fig. 8.

AdvAcc, CorAcc, and CleF1, and shows the most gradual performance gains. This further supports our answer to ①, suggesting that CARD is least constrained by data volume in cross-model TL tasks, maintaining a stable robustness advantage even as data increases. ② StdTrain gains little robustness from more data due to its lack of robust mechanisms. Though TL methods lose some clean performance advantage over StdTrain as data increase, they remain ahead, especially CARD.

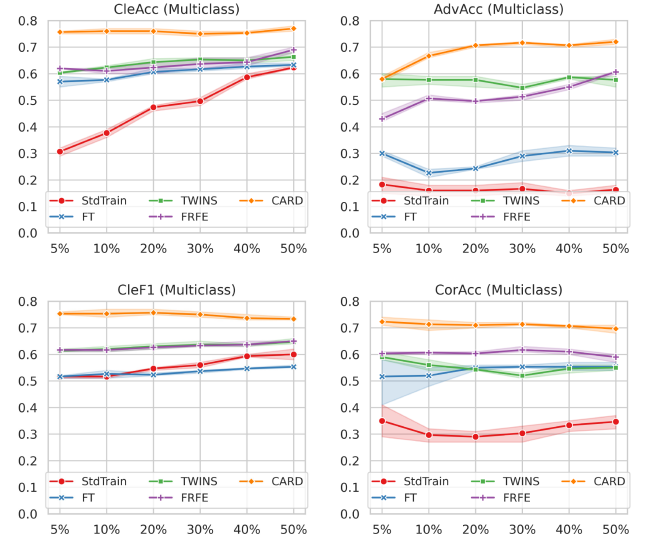


Fig. 10. Impact of target domain training data amount on TL across domains with different input spaces. Generate a 10-class UNSW-NB15-All detector by fine-tuning a robust 5-class NSL-KDD-All detector.

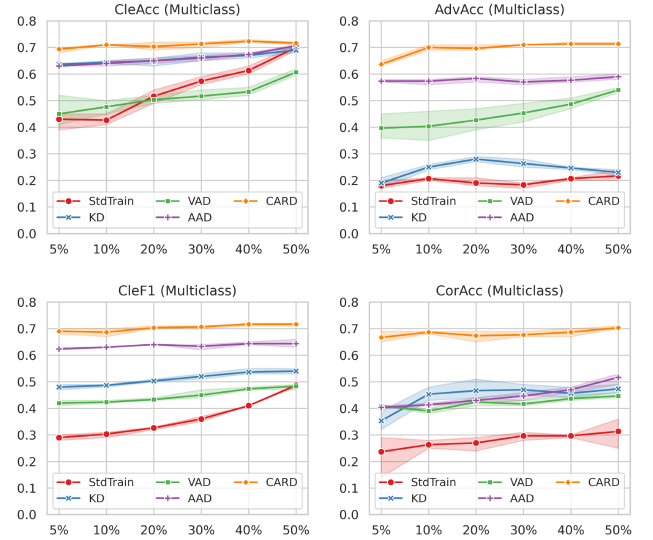


Fig. 11. Impact of target domain training data amount on TL across models with different building blocks. Distill a robust WideResNet-34-10-based 9-class UNSW-NB15-NoExploit detector to MobileNet.

C. Transfer Learning Across Both Domains and Models

Due to the absence of robustness-preserving TL across both data domains and model architectures, we compared CARD was to StdTrain in learning a light multi-class detector from a larger one with different input feature spaces (see Fig. 12).

Results: As data expands, CARD gradually improves in adversarial and natural robustness and clean performance, while StdTrain gains more in clean performance, but their robustness gap remains significant. This confirms that CARD relies less on the amount of target domain training data than StdTrain.

IX. ABLATION STUDY ON THE LOSS FUNCTION

We analyzed how the loss components—Representation Contrastive Loss ($\mathcal{L}_{con}$), Output Distillation Loss ($\mathcal{L}_{dit}$), and Adversarial Classification Loss ($\mathcal{L}_{cla}$)—work independently or

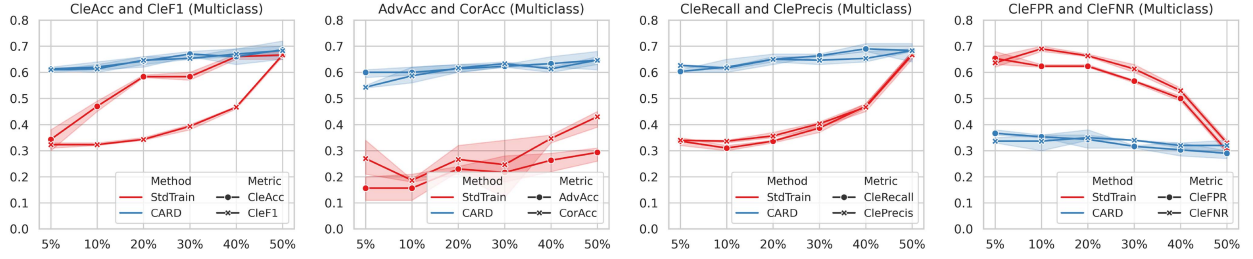


Fig. 12. Impact of target domain training data amount on TL across domains with different input spaces and models with different building blocks. Generate a MobileNet-based 10-class UNSW-NB15-All detector from a WideResNet-34-10-based robust 5-class NSL-KDD-All detector.

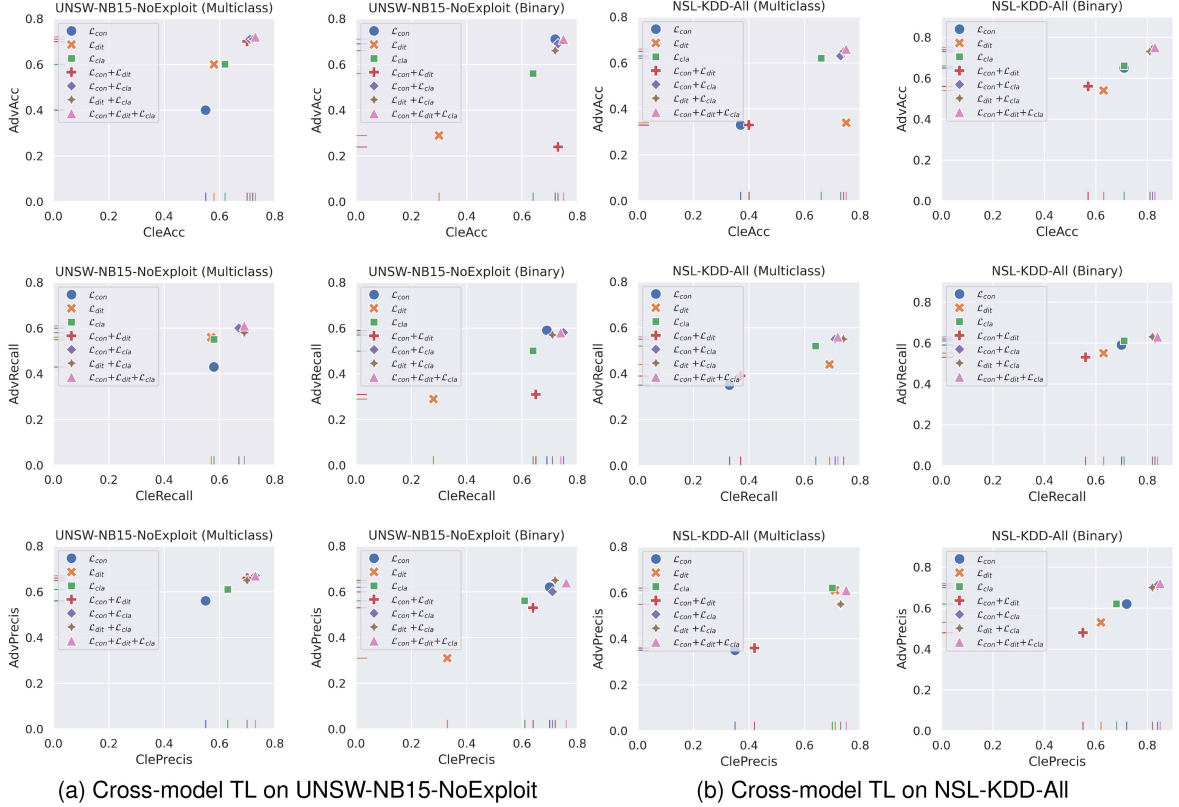


Fig. 13. Ablation study on loss function in TL tasks across models with similar building blocks. (a) Distill robust UNSW-NB15-NoExploit detectors based on WideResNet-34-10 to ResNet-18. (b) Distill robust NSL-KDD-All detectors based on WideResNet-34-10 to ResNet-18.

together in cross-model TL scenarios with 5% data, as all three are used in these tasks (Section V-B). See Fig. 13 for results.

Specifically, employing $\mathcal{L}_{con}$ alone corresponds to applying CRD [47] with enhanced robustness awareness, $\mathcal{L}_{dit}$ alone corresponds to an ensemble of advanced AD methods (VAD [20], RSLAD [9], AAD [22]), and $\mathcal{L}_{cia}$ alone corresponds to adversarial training [23] integrated with standard training.

A. TL Across Model Architectures on UNSW-NB15-NoExploit

See Fig. 13(a) for the results of distilling UNSW-NB15-NoExploit-based robust WideResNet-34-10 into ResNet-18.

Results: ① Using all three losses simultaneously consistently produces the best robustness and clean performance for both multi-class and binary cases. ② In the binary case, $\mathcal{L}_{con}$ is more impactful in robustness and clean performance. In the multi-class case, $\mathcal{L}_{dit}$ and $\mathcal{L}_{cia}$ better improves robustness and

clean performance. ③ $\mathcal{L}_{con}$ alone mainly focuses on hidden representation learning, with limited direct impact on the classification task. However, in the multi-class case, combining it with either $\mathcal{L}_{dit}$ or $\mathcal{L}_{cia}$ greatly improves performance, surpassing each loss individually.

B. TL Across Model Architectures on NSL-KDD-All

See Fig. 13(b) for the results of distilling NSL-KDD-All-based robust WideResNet-34-10 into ResNet-18.

Results: We observed that CARD achieves the best results when all three losses are combined, indicating a stable behavior of our loss function across different datasets. ① Compared to the multi-class case, $\mathcal{L}_{con}$ has a greater impact in the binary case due to the simpler task, making its own contribution to decision boundaries more apparent. ② In the multi-class case, the task's complexity requires combining $\mathcal{L}_{con}$ with other output-based

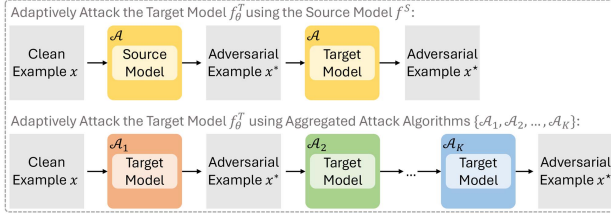


Fig. 14. Adaptive attack strategies.

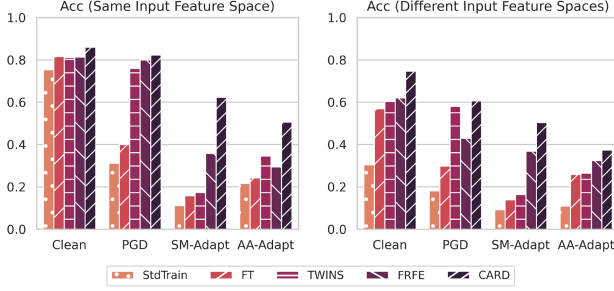


Fig. 15. Robustness against adaptive attacks in cross-domain TL.

losses to enhance classification performance. ③ Combining $\mathcal{L}_{con}$ and $\mathcal{L}_{cla}$ is more effective than combining $\mathcal{L}_{con}$ and $\mathcal{L}_{dit}$, as $\mathcal{L}_{cla}$ introduces ground truth, guiding accurate predictions and balancing robustness with generalization.

X. ADAPTIVE ATTACK AND COMPUTATIONAL COST

In robustness-preserving TL, the hybrid and implicit nature of the target model's robustness origins make it difficult for an attacker, even with access to the white-box target model's parameters, to infer the complete defense algorithm, such as the robust source model's parameters and TL strategy, as detailed in Section IV-A. Nevertheless, considering sophisticated threats that may arise in the real world, we enhanced the adversary's capabilities and conducted adaptive attacks against the target model for further robustness evaluation.

A. Adaptive Attack Against Robustness-Preserving TL

Adaptive attacks, unlike static evasion attacks, dynamically adjust attack strategies to counter defense mechanisms [56], posing a greater threat. We designed two adaptive strategies (see Fig. 14) and evaluated models trained with 30 epochs in the multi-class case (see Figs. 15, 16, and 17).

1) *Source Model-Based Adaptive Attack (SM-Adapt)*: This adaptive strategy assumes the attacker has white-box access to f^S , f_θ^T , training dataset D^T , and knowledge of the perturbation algorithm $\mathcal{A}$ (e.g., PGD) used in the adversarial pretraining of f^S . ① Attacker generates adversarial examples x^* for target domain using f^S and $\mathcal{A}$ with 20 iterations. ② x^* undergoes further perturbation via f_θ^T and $\mathcal{A}$, producing adaptive adversarial examples x^* . ③ In cross-domain TL, due to mismatched input features or label spaces, the attacker must first fine-tune f^S with D^T before generating perturbations.

2) *Aggregated Attacks-Based Adaptive Attack (AA-Adapt)*: This strategy assumes that the attacker has white-box access to f_θ^T and knows that its robustness partly stems from $\mathcal{A}$ (e.g., PGD). The attacker seeks to enhance the attack by sequentially applying attacks $\{\mathcal{A}_1, \dots, \mathcal{A}_K\}$ on x using f_θ^T . We employed

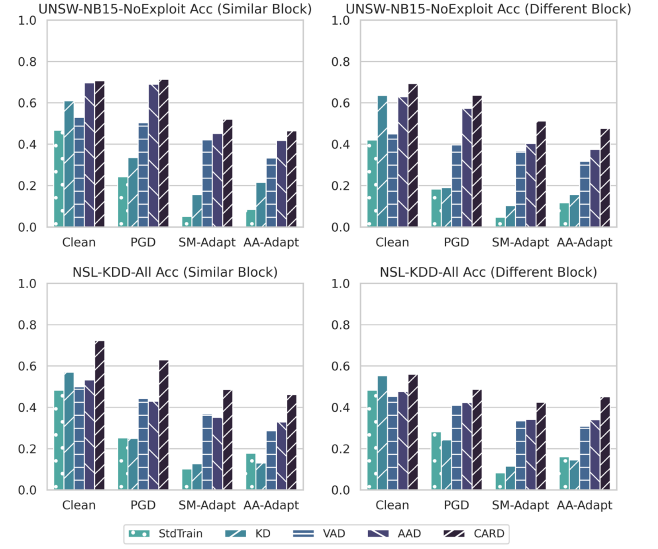


Fig. 16. Robustness against adaptive attacks in cross-model TL.

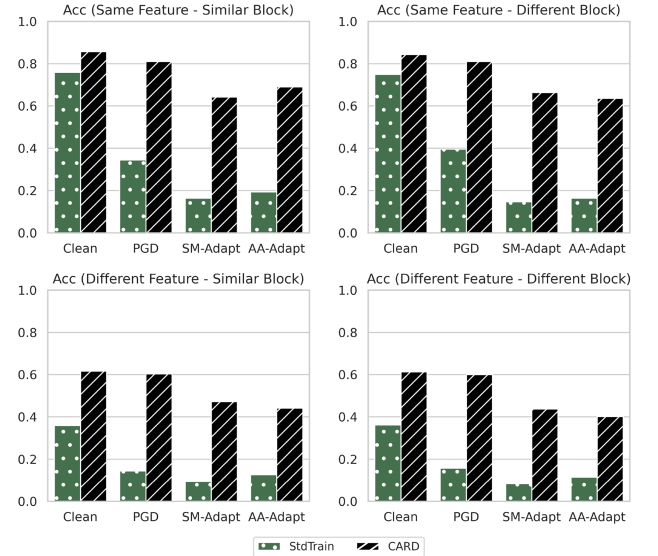


Fig. 17. Robustness against adaptive attacks in cross-domain&model TL.

the AutoAttack [55] framework. In our implementation of $K = 2$ setting, ① attacker first generates x^* using AutoPGD [55] alongside iterations 100. ② x^* are then perturbed using DeepFool [54] with 100 iterations, finally producing x^* .

The results show that CARD consistently outperforms baselines in robustness to adaptive attacks across various TL tasks.

B. Time Cost Comparison on Various TL Tasks

We compared the training time of 50 epochs of CARD and baselines in the data scarcity setting for multi-class detection.

Results: Compared to robustness-preserving TL baselines, CARD trains faster than AAD [22] (2023) and TWINS [21] (2023) but slower than FRFE [6] (2020) and VAD [20] (2020).

- *Cross-domain TL:* Average per-epoch time for StdTrain, FT, TWINS, FRFE, and CARD is 1.13 seconds (s), 1.11 s, 40.01 s, 1.14 s, and 20.58 s, respectively.

- *Cross-model TL*: Average per-epoch time for StdTrain, KD, VAD, AAD, and CARD is 0.35 s, 0.76 s, 3.14 s, 15.68 s, and 4.40 s, respectively.
- *Cross-domain-and-model TL*: Average per-epoch training time for StdTrain and CARD is 0.27 s and 3.15 s, respectively.

Moreover, we found the lighter the target model, the lower the overhead introduced by CARD. Using ResNet or MobileNet as target model reduces time by an average of 80.89% compared to retaining WideResNet from the source model. MobileNet, being the lighter option, trains 20.39% faster on average than ResNet. In summary, CARD outperforms baselines in robustness and clean performance, with a slight time cost over standard TL but greater efficiency than TWINS and AAD.

XI. DISCUSSION OF PRACTICAL DEPLOYMENT

The versatility of CARD across various TL scenarios makes it highly suited for integration into real-world NID systems, offering significant practical benefits. (i) It handles changes in data distribution or feature space, ideal for NID systems in dynamic environments facing evolving network threats. (ii) It generates robust models with limited labeled data, making it suitable for data-scarce systems and reducing reliance on costly training from scratch with abundant labeled data. (iii) It can create efficient target domain models from large source domain models, crucial for deploying NID systems on resource-constrained IoT and mobile devices. (iv) It excels in TL scenarios involving both changes in data domain and model architecture, addressing complex real-world challenges with multiple resource limitations. Overall, CARD is especially valuable for NID practitioners who need to migrate robust detectors in diverse downstream deployment settings.

XII. CONCLUSION

In this paper, we introduce CARD, the first robustness-preserving TL method tailored for the DL-based NID. It produces robust target models against static and adaptive evasion attacks and natural corruptions, using robust source models with limited target domain data. Extensive experiments highlight its superiority in cross-domain (fine-tuning), cross-model (distillation), and other TL tasks involving both domain and model changes. As CARD is not limited to data types, we will explore its applications in image and speech in the future.

ACKNOWLEDGMENT

The authors sincerely thank the editor and all the reviewers for their constructive guidance and valuable feedback.

REFERENCES

- [1] R. U. Khan, X. Zhang, M. Alazab, and R. Kumar, "An improved convolutional neural network model for intrusion detection in networks," in *Proc. IEEE Cybersecurity Cyberforensics Conf.*, 2019, pp. 74–77.
- [2] M. M. Hassan, A. Gumaei, A. Alsanad, M. Alrubaihan, and G. Fortino, "A hybrid deep learning model for efficient intrusion detection in Big Data environment," *Inf. Sci.*, vol. 513, pp. 386–396, 2020.
- [3] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Trans. Emerg. Telecommun. Technol.*, vol. 32, no. 1, 2021, Art. no. e4150.
- [4] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.
- [5] L. Schmidt, S. Santurkar, D. Tsipras, K. Talwar, and A. Madry, "Adversarially robust generalization requires more data," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 5019–5031.
- [6] A. Shafahi et al., "Adversarially robust transfer learning," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–14.
- [7] D. Hendrycks, K. Lee, and M. Mazeika, "Using pre-training can improve model robustness and uncertainty," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 2712–2721.
- [8] B. Wu, J. Chen, D. Cai, X. He, and Q. Gu, "Do wider neural networks really help adversarial robustness?," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 7054–7067.
- [9] B. Zi, S. Zhao, X. Ma, and Y.-G. Jiang, "Revisiting adversarial robustness distillation: Robust soft labels make student better," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 16443–16452.
- [10] S. M. Hamidi and L. Ye, "Adversarial training via adaptive knowledge amalgamation of an ensemble of teachers," 2024, *arXiv:2405.13324*.
- [11] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
- [12] Y. Zhu et al., "Heterogeneous transfer learning for image classification," in *Proc. AAAI Conf. Artif. Intell.*, 2011, pp. 1304–1309.
- [13] S. Bashath, N. Perera, S. Tripathi, K. Manjang, M. Dehmer, and F. E. Streib, "A data-centric review of deep transfer learning with applications to text data," *Inf. Sci.*, vol. 585, pp. 498–528, 2022.
- [14] A. Singla, E. Bertino, and D. Verma, "Preparing network intrusion detection deep learning models with minimal data using adversarial domain adaptation," in *Proc. 15th ACM Asia Conf. Comput. Commun. Secur.*, 2020, pp. 127–140.
- [15] S. T. Mehedi, A. Anwar, Z. Rahman, K. Ahmed, and R. Islam, "Dependable intrusion detection system for IoT: A deep transfer learning based approach," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 1006–1017, Jan. 2023.
- [16] R. Bommasani et al., "On the opportunities and risks of foundation models," 2021, *arXiv:2108.07258*.
- [17] A. Singla, E. Bertino, and D. Verma, "Overcoming the lack of labeled data: Training intrusion detection models using transfer learning," in *Proc. IEEE Int. Conf. Smart Comput.*, 2019, pp. 69–74.
- [18] R. Zhao, Y. Chen, Y. Wang, Y. Shi, and Z. Xue, "An efficient and lightweight approach for intrusion detection based on knowledge distillation," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.
- [19] Z. Wang, Z. Li, D. He, and S. Chan, "A lightweight approach for network intrusion detection in industrial cyber-physical systems based on knowledge distillation and deep metric learning," *Expert Syst. Appl.*, vol. 206, 2022, Art. no. 117671.
- [20] M. Goldblum, L. Fowl, S. Feizi, and T. Goldstein, "Adversarially robust distillation," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 3996–4003.
- [21] Z. Liu, Y. Xu, X. Ji, and A. B. Chan, "TWINS: A fine-tuning framework for improved transferability of adversarial robustness and generalization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 16436–16446.
- [22] B. Huang, M. Chen, Y. Wang, J. Lu, M. Cheng, and W. Wang, "Boosting accuracy and robustness of student models via adaptive adversarial distillation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 24668–24677.
- [23] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–23.
- [24] G. Andresini, A. Appice, C. Loglisci, V. Belvedere, D. Redavid, and D. Malerba, "A network intrusion detection system for concept drifting network traffic data," in *Proc. Int. Conf. Discov. Sci.*, 2021, pp. 111–121.
- [25] Y. Xu et al., "Intrusion detection based on fusing deep neural networks and transfer learning," in *Proc. Int. Forum Digit. TV Wireless Multimedia Commun.*, 2020, pp. 212–223.
- [26] R. Müller, S. Kornblith, and G. E. Hinton, "When does label smoothing help?," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 4694–4703.
- [27] B. Heo, M. Lee, S. Yun, and J. Y. Choi, "Knowledge transfer via distillation of activation boundaries formed by hidden neurons," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 3779–3787.
- [28] T. Gao, X. Jin, and Y. Lai, "Model heterogeneous federated learning for intrusion detection based on knowledge distillation," in *Proc. Int. Conf. Commun. Netw. Secur.*, 2022, pp. 30–35.
- [29] L. Fan, S. Liu, P.-Y. Chen, G. Zhang, and C. Gan, "When does contrastive learning preserve adversarial robustness from pretraining to finetuning?," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 80–92.

- [30] A. Muhammad, F. Zhou, C. Xie, J. Li, S. Bae, and Z. Li, "MixACM: Mixup-based robustness transfer via distillation of activated channel maps," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 55–69.
- [31] J. Zhu et al., "Reliable adversarial distillation with unreliable teachers," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–15.
- [32] J. Maroto, G. Ortiz-Jiménez, and P. Frossard, "On the benefits of knowledge distillation for adversarial robustness," 2022, *arXiv:2203.07159*.
- [33] T. Bai, J. Zhao, and B. Wen, "Guided adversarial contrastive distillation for robust students," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 9643–9655, 2023.
- [34] M. Kim, J. Tack, and S. J. Hwang, "Adversarial self-supervised contrastive learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 2983–2994.
- [35] L. F. Nern, H. Raj, M. A. Georgi, and Y. Sharma, "On transfer of adversarial robustness from pretraining to downstream tasks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 59206–59226.
- [36] Z. Jiang, T. Chen, T. Chen, and Z. Wang, "Robust pre-training by adversarial contrastive learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 16199–16210.
- [37] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
- [38] K. Wang et al., "BARS: Local robustness certification for deep learning based traffic analysis systems," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.
- [39] M. Huang et al., "ARIoTDef: Adversarially robust IoT early defense system based on self-evolution against multi-step attacks," *ACM Trans. Internet Things*, vol. 5, no. 3, pp. 1–34, 2024.
- [40] M. Huang, Y. Lin, X. Chen, and E. Bertino, "Dimensional robustness 1281 certification for deep neural networks in network intrusion detection systems," *ACM Trans. Privacy Secur.*, pp. 1–33, 2025, doi: [10.1145/3715121](https://doi.org/10.1145/3715121).
- [41] E. Mintun, A. Kirillov, and S. Xie, "On interaction between augmentations and corruptions in natural corruption robustness," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 3571–3583.
- [42] J.-M. Lemerrier, J. Richter, S. Welker, and T. Gerkmann, "StoRM: A diffusion-based stochastic regeneration model for speech enhancement and dereverberation," *IEEE/ACM Trans. Audio Speech Lang. Process.*, vol. 31, pp. 2724–2737, 2023.
- [43] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
- [44] J. Li, C. Xiong, and S. C. Hoi, "Learning from noisy data with robust representation learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 9485–9494.
- [45] N. Wang, S. Shi, Y. Chen, W. Lou, and Y. T. Hou, "FeCo: Boosting intrusion detection capability in IoT networks via contrastive learning," *IEEE Trans. Dependable Secure Comput.*, pp. 1–16, 2025, doi: [10.1109/TDSC.2025.3544106](https://doi.org/10.1109/TDSC.2025.3544106). [Online]. Available: <https://ieeexplore.ieee.org/document/10896851>
- [46] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9729–9738.
- [47] Y. Tian, D. Krishnan, and P. Isola, "Contrastive representation distillation," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–19.
- [48] P. Awasthi, N. Dikkala, and P. Kamath, "Do more negative samples necessarily hurt in contrastive learning?," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 1101–1116.
- [49] UNB, "NSL-KDD dataset," 2009. [Online]. Available: <https://www.unb.ca/cic/datasets/nsl.html>
- [50] UNSW, "UNSW-NB15," 2015. [Online]. Available: <https://research.unsw.edu.au/projects/unswnb15-dataset>
- [51] S. Zagoruyko and N. Komodakis, "Wide residual networks," 2016, *arXiv:1605.07146*.
- [52] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [53] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
- [54] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "DeepFool: A simple and accurate method to fool deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2574–2582.
- [55] F. Croce and M. Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 2206–2216.

- [56] F. Tramer, N. Carlini, W. Brendel, and A. Madry, "On adaptive attacks to adversarial example defenses," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 1633–1645.



Mengdie Huang (Student Member, IEEE) received the MS degree in electronic information engineering from the Communication University of China, Beijing, China, in 2019. She is currently working toward the PhD degree in cyberspace security from Xidian University, Xi'an, China. She has been visiting the Department of Computer Science at Purdue University, West Lafayette, since 2022. Her research interests are in machine learning security, network security, and multimedia security.



Yingjun Lin received the BS degree in data science from Purdue University, West Lafayette, in 2024. He is currently working toward the MS degree in computer science with the Georgia Institute of Technology. His research interests include machine learning security, generative AI, large language models, and computer vision.



Ninghui Li (Fellow, IEEE) received the bachelor's degree from the University of Science and Technology of China, in 1993, and the PhD degree in computer science from New York University, in 2000. He is a Samuel D. Conte professor of computer science with Purdue University. His research interests are in security and privacy, and he has published more than 200 papers. He is editor-in-chief for *ACM Transactions on Privacy and Security* since 2020, and has served as program chair for several conferences, including ACM CCS, ESORICS, ACM ASIACCS, ACM SACMAT, etc. He served as chair of ACM SIGSAC from 2017 to 2021, and vice chair from 2013 to 2017.



Xiaofeng Chen (Fellow, IEEE) received the BS and MS degrees in mathematics from Northwest University, China, in 1998 and 2000, respectively, and the PhD degree in cryptography from Xidian University, in 2003. Currently, he works with Xidian University as a professor. His research interests include applied cryptography and cloud computing security. He has published more than 300 research papers. His work has been cited more than 16000 times with Google Scholar. He is in the editorial board of *IEEE Transactions on Dependable and Secure Computing* and *IEEE Transactions on Knowledge and Data Engineering*, *International Journal of Foundations of Computer Science*, etc. He has served as the program/general chair or program committee member in more than 30 conferences.



She is a fellow member of ACM and AAAS.

Elisa Bertino (Fellow, IEEE) is a Samuel D. Conte distinguished professor of computer science with Purdue University. She serves as director of the Purdue Cyberspace Security Lab, and leads research in data security and privacy. Before joining Purdue, in 2004, she was a professor and department head with the Department of Computer Science and Communication, University of Milan. She has been a postdoc researcher with the IBM Research Laboratory in San Jose, and a visiting professor with Singapore Management University and Singapore National University.